

LAPORAN TUGAS 1 IF4073 PEMROSESAN CITRA DIGITAL

Analisis dan Perbaikan Kualitas Citra



Disusun oleh:

Abrar Abhirama Widyadhana – 13523038

Jethro Jens Norbert Simatupang – 13523081

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2026**

Pernyataan Tidak Melakukan Kecurangan

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (*raw output*) dari *Generative AI*, melainkan hasil pengembangan dan penulisan mandiri.

[Tanda tangan Mahasiswa 1]
Abrar Abhirama Widyadhana

[Tanda tangan Mahasiswa 2]
Jethro Jens Norbert Simatupang

DAFTAR ISI

Pernyataan Tidak Melakukan Kecurangan.....	2
DAFTAR ISI.....	3
BAB 1	
PENDAHULUAN.....	4
1.1 Deskripsi Singkat Program.....	4
BAB 2	
DASAR TEORI.....	5
2.1 Histogram Citra.....	5
2.2 Intensity Transformation.....	5
2.3 Histogram Equalization.....	7
2.5 Image Filtering dengan Masking.....	8
BAB 3	
PERANCANGAN DAN IMPLEMENTASI.....	10
3.1 Struktur Program.....	10
3.2 Alur Penggunaan GUI.....	10
3.3 Implementasi Fungsi Histogram.....	12
3.4 Implementasi Intensity Transformation.....	14
3.5 Implementasi Histogram Equalization.....	17
3.6 Implementasi Histogram Matching.....	21
3.7 Implementasi Image Filtering.....	25
BAB IV	
PENGUJIAN DAN ANALISIS.....	26
4.1 Skenario Pengujian.....	26
4.2 Pengujian Histogram Citra.....	26
4.3 Pengujian Kasus 1.....	27
4.4 Pengujian Kasus 2.....	31
4.5 Pengujian Kasus 3.....	35
4.6 Pengujian Kasus 4.....	39
BAB V	
PENUTUP.....	45
5.1 Kesimpulan.....	45
5.2 Saran.....	45
DAFTAR PUSTAKA.....	46
LAMPIRAN.....	47

BAB 1

PENDAHULUAN

1.1 Deskripsi Singkat Program

Program ini merupakan aplikasi berbasis MATLAB yang dirancang untuk melakukan perbaikan kualitas citra (*image enhancement*) melalui antarmuka grafis (*Graphical User Interface/GUI*). Aplikasi ini menyediakan beberapa metode *image enhancement* yang dapat dipilih sesuai dengan karakteristik dan permasalahan visual pada citra.

Metode *image enhancement* yang diimplementasikan terdiri atas empat kelompok utama:

- a. Intensity Transformation, yaitu transformasi nilai intensitas piksel untuk melakukan penyesuaian kecerahan dan kontras.
- b. Histogram Equalization, yaitu teknik pengolahan histogram untuk meningkatkan kontras dengan mendistribusikan intensitas citra secara lebih merata.
- c. Histogram Specification/Matching, yaitu teknik penyesuaian distribusi intensitas citra agar mendekati histogram dari citra referensi atau histogram target tertentu.
- d. Image Filtering dengan Masking, yaitu teknik pemrosesan spasial menggunakan *mask* atau *kernel*, baik berupa *linear filtering* maupun *non-linear filtering*, untuk mengurangi noise, menghaluskan citra, atau meningkatkan ketajaman dan tepi.

Melalui GUI, pengguna dapat:

- memuat citra input dari file;
- memilih metode *enhancement* yang akan digunakan;
- mengatur parameter metode secara interaktif;
- melihat citra sebelum dan sesudah *enhancement*;
- melihat dan membandingkan histogram citra sebelum dan sesudah *enhancement*; dan
- menganalisis distribusi intensitas citra melalui histogram RGB pada masing-masing kanal warna.

Implementasi algoritma inti dilakukan menggunakan fungsi yang dibuat sendiri. Fungsi bawaan MATLAB seperti *histeq*, *imhistmatch*, *imfilter*, dan *medfilt2* tidak digunakan dalam implementasi utama metode, sehingga proses perhitungan histogram dan *enhancement* dapat dijelaskan dan dikendalikan secara eksplisit.

BAB 2

DASAR TEORI

2.1 Histogram Citra

Histogram citra adalah representasi grafik distribusi frekuensi intensitas piksel dalam suatu citra digital.

Untuk citra grayscale 8-bit, histogram didefinisikan sebagai:

$$h(r_k) = n_k$$

dengan:

- r_k = nilai intensitas ke- k , dengan $k = 0, 1, 2, \dots, 255$
- n_k = jumlah piksel dengan intensitas r_k

Histogram dapat dinormalisasi menjadi *Probability Density Function* (PDF):

$$p(r_k) = \frac{n_k}{N \times M}$$

dengan $N \times M$ merupakan dimensi citra atau jumlah total piksel.

CDF merupakan distribusi kumulatif dari PDF:

$$c(r_k) = \sum_{j=0}^k p(r_j) = \frac{1}{N \times M} \sum_{j=0}^k n_j$$

Untuk citra berwarna (RGB), histogram dihitung secara terpisah untuk setiap kanal warna, yaitu merah (R), hijau (G), dan biru (B). Setiap kanal memiliki 256 tingkat intensitas.

2.2 Intensity Transformation

Intensity Transformation adalah operasi *point-wise* yang mentransformasi setiap piksel input r menjadi piksel output s berdasarkan fungsi transformasi:

$$s = T(r)$$

a. Brightness Adjustment

Brightness adjustment digunakan untuk menaikkan atau menurunkan tingkat kecerahan citra dengan menambahkan nilai offset pada setiap piksel. Rumus yang digunakan adalah:

$$s = r + \text{brightness} \times 255$$

Parameter brightness berada pada rentang relatif terhadap nilai maksimum intensitas. Nilai positif meningkatkan kecerahan, sedangkan nilai negatif menurunkan kecerahan.

b. Contrast Correction

Contrast correction digunakan untuk meningkatkan atau menurunkan perbedaan intensitas piksel terhadap nilai rata-rata citra. Rumus yang digunakan adalah:

$$s = c(r - \mu) + \mu$$

dengan:

- r = intensitas piksel input

- s = intensitas piksel output
- μ = nilai rata-rata intensitas citra
- c = faktor contrast

c. Negasi Citra (*Negative*)

Negasi citra membalik nilai intensitas piksel:

$$s = 255 - r$$

Transformasi ini menghasilkan efek seperti *film negatif*, dengan nilai 0 menjadi 255, nilai 128 menjadi 127, dan seterusnya. Metode ini dapat digunakan untuk meningkatkan keterlihatan detail pada region gelap.

d. Transformasi Log

Transformasi log melakukan kompresi *dynamic range*:

$$s = c \times \frac{\log(1 + r)}{\log(256)} \times 255$$

Karena nilai maksimum $\log(1 + 255) = \log(256)$, normalisasi dilakukan dengan membagi $\log(256)$ agar output dapat mengisi rentang $[0, 255]$.

Sifat-sifat transformasi log:

- Memperjelas region gelap (*dark regions*).
- Mempersempit region terang.
- Berguna untuk citra dengan *dynamic range* yang tinggi.

Parameter c :

- $c = 1$: *Mapping* standar.
- $c > 1$: Hasil lebih cerah.
- $c < 1$: Hasil lebih gelap.

e. Transformasi Power-Law (*Gamma Correction*)

Transformasi *power-law* atau *gamma correction* didefinisikan sebagai:

$$s = c \times \left(\frac{r}{255}\right)^\gamma \times 255$$

Nilai input terlebih dahulu dinormalisasi ke rentang $[0, 1]$, kemudian dipangkatkan dengan γ , dan hasilnya dikalikan kembali dengan 255.

Parameter γ :

- $\gamma < 1$: Memperjelas region gelap.
- $\gamma > 1$: Memperjelas region terang.
- $\gamma = 1$: Tidak ada perubahan.

f. Contrast Stretching

Contrast stretching memetakan rentang intensitas aktual $[r_{\min}, r_{\max}]$ ke rentang penuh

$[0, 255]$:

$$s = \frac{r - r_{\min}}{r_{\max} - r_{\min}} \times 255$$

Citra dengan kontras rendah akan mengalami peningkatan kontras setelah transformasi karena rentang intensitasnya diregangkan hingga memenuhi rentang $[0, 255]$.

2.3 Histogram Equalization

Histogram Equalization adalah teknik peningkatan kontras dengan mentransformasi distribusi intensitas citra agar lebih merata menggunakan CDF (*Cumulative Distribution Function*) sebagai fungsi transformasi.

Prosedur:

1. Hitung histogram:

$$h(r_k) = n_k$$

2. Hitung PDF:

$$p(r_k) = \frac{n_k}{N \times M}$$

3. Hitung CDF:

$$s_k = \frac{1}{N \times M} \sum_{j=0}^k n_j$$

4. Normalisasi CDF:

$$\text{CDF}_{\text{norm}}(k) = \text{round}(255 \times s_k)$$

5. Terapkan mapping:

$$s = \text{CDF}_{\text{norm}}(r)$$

Variasi untuk Citra Warna

- a. Global: Equalisasi setiap kanal secara independen.
- b. HSV: Equalisasi hanya pada channel Value.
- c. YCbCr: Equalisasi hanya pada channel Y.

Untuk HSV:

$$V = \max(R, G, B)$$

$$S = \frac{V - \min(R, G, B)}{V}$$

Untuk YCbCr berdasarkan standar ITU-R BT.601:

$$Y = 0.299R + 0.587G + 0.114B$$

$$Cb = 128 - 0.168736R - 0.331264G + 0.5B$$

$$Cr = 128 + 0.5R - 0.418688G - 0.081312B$$

2.4 Histogram Specification / Histogram Matching

Histogram Matching atau Histogram Specification adalah teknik untuk mentransformasi citra agar distribusi intensitasnya mendekati histogram citra referensi.

Diberikan:

- $F(r) = \text{CDF citra input.}$

- $G(z) = \text{CDF citra referensi.}$

Tujuannya adalah memetakan intensitas input ke intensitas yang sesuai dengan distribusi referensi.

Prosedur

1. Hitung CDF citra input:

$$F(r_k) = \sum_{j=0}^k p_r(r_j)$$

2. Hitung CDF citra referensi:

$$G(z_k) = \sum_{j=0}^k p_z(z_j)$$

3. Buat *Look-Up Table* (LUT):

$$\text{LUT}[r] = |G(z) - F(r)|$$

Nilai z dipilih berdasarkan CDF referensi yang paling dekat dengan $F(r)$.

4. Terapkan LUT:

$$s = \text{LUT}[r]$$

2.5 Image Filtering dengan Masking

Image Filtering adalah proses mengolah citra berdasarkan nilai piksel dan piksel tetangganya (*neighborhood*) menggunakan *mask* atau *kernel*.

Kernel merupakan matriks numerik yang menentukan pengaruh piksel di sekitar suatu piksel.

Konvolusi 2D didefinisikan sebagai:

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x, +sy + t)$$

dengan w adalah kernel, f citra input, g citra output, dan a, b radius kernel.

Untuk menangani piksel di luar batas citra digunakan *padding*:

- a. replicate: Menduplikasi piksel tepi.
- b. zero: Menggunakan piksel bernilai 0.
- c. symmetric: Mencerminkan piksel tepi.

Dua kelompok filtering

- a. Linear Filtering

Linear filtering menggunakan kombinasi linear piksel dalam *neighborhood*.

- Mean Filter digunakan untuk *smoothing* dan pengurangan noise:

$$w = \frac{1}{k^2} \mathbf{1}_{k \times k}$$

- Gaussian Filter memberikan bobot lebih besar pada piksel di sekitar pusat:

$$w(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

Parameter σ menentukan tingkat *blur*.

- Sharpen Filter digunakan untuk meningkatkan ketajaman tepi:

$$w_4 = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad w_8 = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

- Laplacian Filter mendeteksi tepi berdasarkan turunan kedua:

$$w_4 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad w_8 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- Sobel Filter mendeteksi tepi berdasarkan gradien intensitas:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Magnitudo gradien:

$$G = \sqrt{G_x^2 + G_y^2}$$

b. Non-Linear Filtering

Median Filter merupakan filter non-linear yang efektif untuk menghilangkan *salt-and-pepper noise*:

$$g(x, y) = \text{median}\{f(x, +sy + t) \mid (s, t) \in W\}$$

Prosesnya meliputi pengambilan *window*, pengurutan nilai piksel, pengambilan nilai median, dan penempatan hasil pada piksel output.

Karakteristik Median Filter:

- Efektivitas: Efektif untuk *salt-and-pepper noise*.
- Edge Preservation: Mempertahankan tepi lebih baik daripada *linear smoothing*.
- Window Size: Umumnya 3×3 , 5×5 , atau 7×7 .
- Kompleksitas: Lebih lambat karena memerlukan pengurutan nilai piksel.

BAB 3

PERANCANGAN DAN IMPLEMENTASI

3.1 Struktur Program

Program disusun dalam struktur folder yang terorganisir berdasarkan fungsionalitas. Berikut struktur direktori utama:

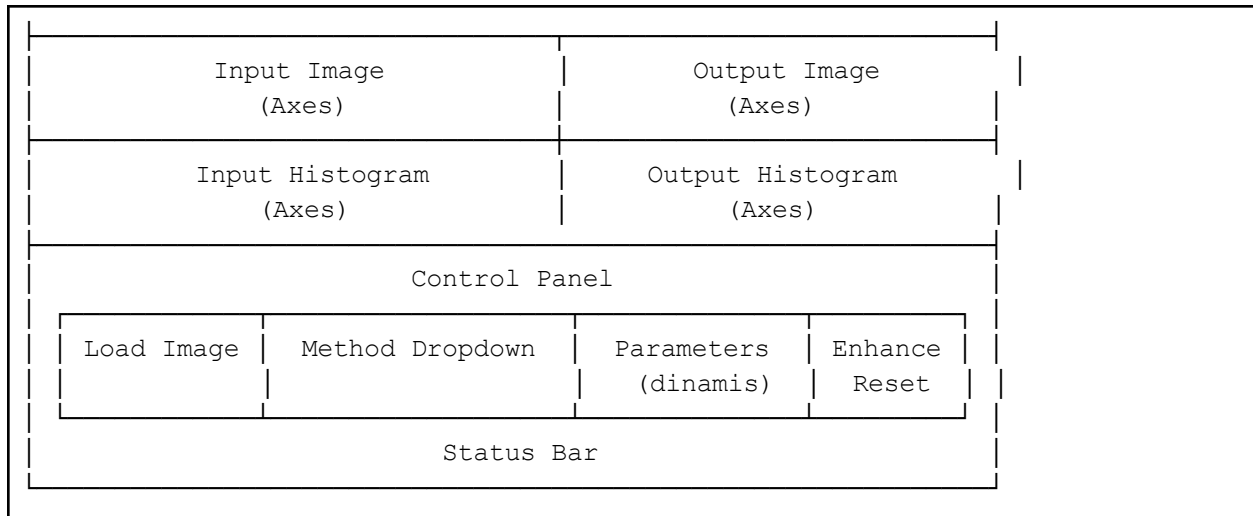
```
ImageEnhancement-GUI/
├── src/
│   ├── analysis/                # Modul analisis citra
│   │   ├── histogram/          # Fungsi histogram
│   │   │   ├── computeCDF.m
│   │   │   ├── computeHistogram.m
│   │   │   ├── computeHistogramGrayscale.m
│   │   │   └── computeHistogramRGB.m
│   │   └── analyzeImage.m
│   ├── enhancement/            # Modul teknik enhancement
│   │   ├── IntensityTransformation/
│   │   │   └── intensityTransform.m
│   │   ├── HistogramEqualization/
│   │   │   └── histogramEqualization.m
│   │   ├── HistogramSpecification/
│   │   │   └── *.m
│   │   ├── Filtering/
│   │   │   └── *.m
│   │   └── applyEnhancement.m
│   ├── gui/                    # Antarmuka pengguna
│   │   ├── ImageEnhancementApp.m
│   │   └── imageEnhancementGUI.m
│   ├── utils/                  # Fungsi utilitas
│   │   ├── computeHistogramChannel.m
│   │   ├── computeHistogramLocal.m
│   │   ├── computeImageStats.m
│   │   └── visualizeEnhancementComparison.m
│   └── test/                   # Skrip pengujian
├── startup.m                   # Inisialisasi path
└── README.md
```

Program dirancang dengan prinsip modularitas, di mana setiap teknik enhancement diimplementasikan sebagai fungsi terpisah. Hal ini memungkinkan *reusability*, *testability*, *maintainability*, dan *extensibility*.

3.2 Alur Penggunaan GUI

GUI terdiri dari empat area utama:

IMAGE ENHANCEMENT APP



Langkah-langkah penggunaan:

1. Memuat Citra
 - Klik "Load Image" dan pilih file citra.
 - Citra dan histogram input akan ditampilkan secara otomatis.
2. Memilih Metode
 - Pilih metode pada dropdown "Method":
 - Intensity Transformation
 - Histogram Equalization
 - Histogram Matching
 - Linear Filtering
 - Median Filtering
 - Panel Parameters akan menampilkan sub-metode dan parameter yang sesuai.
3. Mengatur Parameter
 - Pilih sub-metode dan atur parameter melalui slider atau edit field.
4. Menjalankan Enhancement
 - Klik "Enhance".
 - Citra hasil dan histogram output akan ditampilkan, serta status metode dan parameter.
5. Menyimpan atau Menyalin Hasil
 - Pada gambar input atau hasil, klik ikon tiga titik yang ada di atas gambar.
 - Pilih "Save" untuk menyimpan gambar ke perangkat.

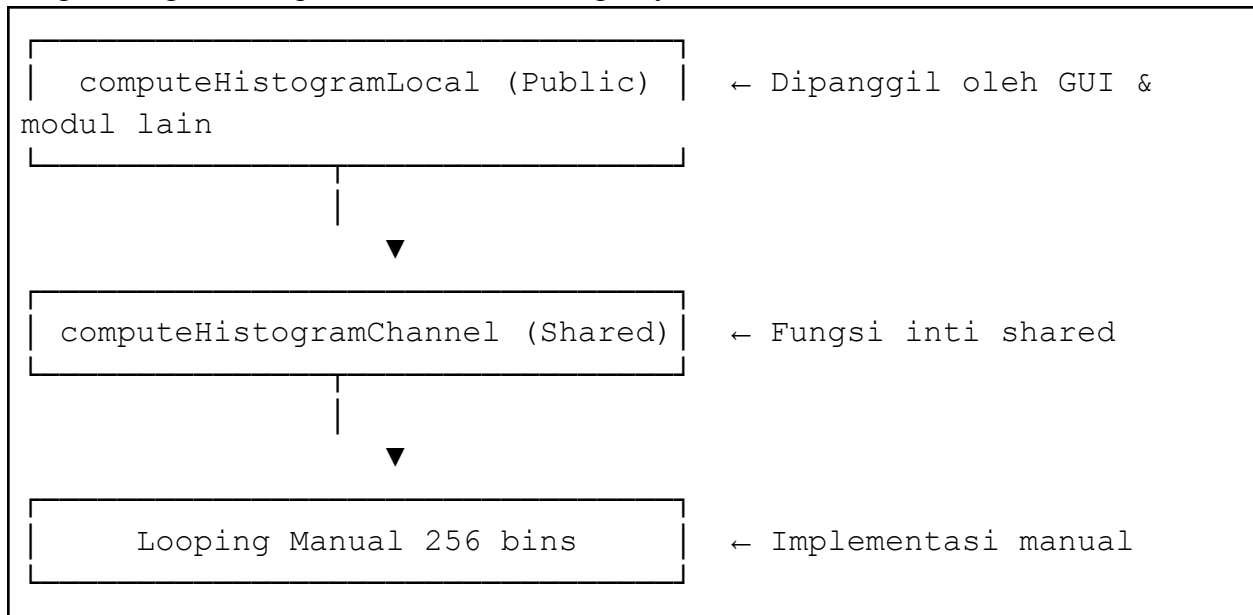
- Pilih "Copy" untuk menyalin gambar ke *clipboard* sehingga dapat ditempelkan ke aplikasi lain.

6. Reset atau Ulangi

- Klik "Reset" untuk kembali ke kondisi awal.
- Ubah metode atau parameter untuk melakukan percobaan lain.

3.3 Implementasi Fungsi Histogram

Fungsi histogram diimplementasikan dalam tiga layer:



a. Fungsi Utama: computeHistogramLocal

Fungsi ini adalah entry point utama yang dipanggil oleh GUI dan modul lainnya.

```

% computeHistogramLocal.m
function varargout = computeHistogramLocal(img)
    % Validasi input
    if ~isnumeric(img)
        error('Input must be numeric array');
    end

    isColored = (size(img, 3) == 3);

    if isColored
        % Histogram RGB - ekstrak dan hitung setiap channel
        r = double(img(:, :, 1));
        g = double(img(:, :, 2));
        b = double(img(:, :, 3));

        countsR = computeHistogramChannel(r);
        countsG = computeHistogramChannel(g);
  
```



```

countsB = computeHistogramChannel(b);

varargout{1} = countsR;
varargout{2} = countsG;
varargout{3} = countsB;
else
% Histogram grayscale
imgDouble = double(img);
counts = computeHistogramChannel(imgDouble);

varargout{1} = counts;
varargout{2} = []; % Empty untuk menandakan grayscale
varargout{3} = [];
end
end

```

Karakteristik:

- Mendukung citra grayscale dan RGB
- Untuk RGB, histogram dihitung terpisah per kanal
- Mengembalikan 3 output: countsR, countsG, countsB
- Untuk grayscale, output kedua dan ketiga kosong (empty array)

b. Fungsi Shared: computeHistogramChannel

Fungsi ini adalah implementasi inti yang menghitung histogram untuk satu channel.

```

% computeHistogramChannel.m
function counts = computeHistogramChannel(channel)
% Konversi ke double dan bulatkan
ch = double(channel);
ch = round(ch);
ch = max(0, min(255, ch)); % Clamp ke range [0, 255]

[m, n] = size(ch);

% Inisialisasi histogram 256 bins (0-255)
counts = zeros(256, 1);

% Hitung frekuensi setiap level intensitas
for i = 1:m
for j = 1:n
% Index = intensitas + 1 (karena MATLAB 1-based indexing)
idx = ch(i, j) + 1;
idx = max(1, min(256, idx)); % Clamp untuk safety
counts(idx) = counts(idx) + 1;
end
end
end

```

Prosedur:

1. Konversi input ke tipe double

2. Bulatkan nilai (rounding) untuk akurasi indexing
3. Clamp nilai ke range [0, 255]
4. Inisialisasi array kosong 256×1
5. Iterasi setiap piksel dan increment counter yang sesuai

Konvensi Indexing:

- Nilai intensitas 0 $\rightarrow$ index 1
- Nilai intensitas 255 $\rightarrow$ index 256
- Menggunakan +1 karena MATLAB menggunakan 1-based indexing

Fungsi histogram yang dibuat digunakan kembali oleh:

Modul	Penggunaan
GUI	Menampilkan histogram input/output
Histogram Equalization	Menghitung histogram untuk mapping CDF
Histogram Matching	Menghitung histogram input dan referensi
computeImageStat	Menghitung entropi citra

3.4 Implementasi Intensity Transformation

Fungsi `intensityTransform` adalah fungsi utama yang menjadi interface untuk semua jenis transformasi intensitas.

```
% intensityTransform.m
function result = intensityTransform(img, varargin)
    p = inputParser;
    addRequired(p, 'img', @isnumeric);
    addRequired(p, 'transformType', @ischar);
    addParameter(p, 'c', 1);
    addParameter(p, 'gamma', 1);
    addParameter(p, 'offset', 0.1);
    parse(p, img, varargin{1}, varargin{2:end});

    img = double(img);

    switch lower(p.Results.transformType)
    case 'negative'
        result = transformNegative(img);
    case 'log'
        result = transformLog(img, p.Results.c);
    case 'power'
        result = transformPower(img, p.Results.c, p.Results.gamma);
    case 'contrast'
        result = transformContrastStretching(img);
    case 'histogram_slide'
        result = transformHistogramSlide(img, p.Results.offset);
```

```

        case 'histogram_stretch'
            result = transformHistogramStretch(img);
        end

        result = uint8(result);
    end

```

Parameter:

img	Citra input (grayscale atau RGB)
transformType	Jenis transformasi ('negative', 'log', 'power', 'contrast', dll.)
c	Konstanta pengali (default: 1)
gamma	Eksponen untuk power-law (default: 1)
offset	Pergeseran untuk histogram slide (default: 0.1)

a. Brightness Adjustment

```

% Brightness Adjustment: s = r + brightness * 255
function result = transformBrightnessAdjustment(img, brightness)
    brightnessVal = brightness * 255;
    result = img + brightnessVal;
    result = max(0, min(255, result));
end

```

brightnessVal mengubah parameter brightness menjadi nilai intensitas aktual. Setelah transformasi, nilai piksel dibatasi agar tetap berada pada rentang [0,255].

b. Contrast Correction

```

% Contrast Correction: s = c * (r - mean) + mean
function result = transformContrastCorrection(img, contrast)
    if size(img, 3) == 3
        result = zeros(size(img));
        for ch = 1:3
            channel = img(:, :, ch);
            meanVal = mean(channel(:));
            result(:, :, ch) = contrast * (channel - meanVal) +
meanVal;
        end
    else
        meanVal = mean(img(:));
        result = contrast * (img - meanVal) + meanVal;
    end
end

```

```

        result = max(0, min(255, result));
    end

```

meanVal digunakan sebagai titik pusat sehingga perubahan kontras memperbesar atau memperkecil jarak setiap piksel terhadap rata-rata, tanpa secara langsung menggeser nilai rata-rata citra.

c. Negasi Citra

```

function result = transformNegative(img)
    if size(img, 3) == 3
        result = zeros(size(img));
        for c = 1:3
            result(:, :, c) = 255 - img(:, :, c);
        end
    else
        result = 255 - img;
    end
end

```

Mendukung citra grayscale dan RGB dengan formula: $s = 255 - r$.

d. Transformasi Log

```

function result = transformLog(img, c)
    result = c * (log(1 + img) / log(256)) * 255;
    result = max(0, min(255, result));
end

```

Formula: $s = c \cdot \frac{\log(1+r)}{\log(256)} \cdot 255$

Penjelasan normalisasi:

- Nilai maksimum $\log(256) \approx 5.55$ terjadi saat $r = 255$
- Pembagian dengan $\log(256)$ memastikan output $\max = c \times 255$
- Parameter c mengontrol kecerahan hasil

e. Transformasi Power-Law

```

function result = transformPower(img, c, gamma)
    result = c * ((img / 255) .^ gamma) * 255;
    result = max(0, min(255, result));
end

```

Formula: $s = c \cdot \left(\frac{r}{255}\right)^\gamma \cdot 255$

Penjelasan:

- Input dinormalisasi ke $[0, 1]$ dengan pembagian 255
- Diberi pangkat γ untuk kurva
- Dikali 255 untuk kembali ke range original
- Parameter c mengontrol kecerahan, γ mengontrol shape kurva

f. Contrast Stretching

```

function result = transformContrastStretching(img)
    if size(img, 3) == 3
        result = zeros(size(img));
        for c = 1:3
            ch = img(:, :, c);
            mn = min(ch(:));
            mx = max(ch(:));
            if mx > mn
                result(:, :, c) = (ch - mn) / (mx - mn) * 255;
            else
                result(:, :, c) = ch;
            end
        end
    else
        mn = min(img(:));
        mx = max(img(:));
        if mx > mn
            result = (img - mn) / (mx - mn) * 255;
        else
            result = img;
        end
    end
end

```

Formula: $s = \frac{r - r_{min}}{r_{max} - r_{min}} \cdot 255$

Penjelasan:

- Mencari nilai minimum dan maksimum aktual
- Memetakan range tersebut ke [0, 255]
- Untuk RGB, setiap channel diproses terpisah

3.5 Implementasi Histogram Equalization

Fungsi Utama: histogramEqualization

```

% histogramEqualization.m
function result = histogramEqualization(img, varargin)
    % Parse input
    p = inputParser;
    addRequired(p, 'img', @isnumeric);
    addParameter(p, 'mode', 'global');
    parse(p, img, varargin{:});

    mode = lower(p.Results.mode);
    isColored = (size(img, 3) == 3);

    if isColored
        switch mode
            case 'global'

```

```

        result = equalizeGlobal(img);
    case 'hsv'
        result = equalizeHSV(img);
    case 'ycbcr'
        result = equalizeYCbCr(img);
    end
else
    result = equalizeGrayscale(img);
end

result = uint8(result);
end

```

Mendukung tiga mode: `global` (equalize setiap channel secara independen), `hsv` (equalize hanya channel Value), dan `ycbcr` (equalize hanya channel Luminance (Y))

a. Histogram Equalization Grayscale

```

function result = equalizeGrayscale(img)
    [m, n] = size(img);
    numPixels = m * n;
    img = double(img);

    % Hitung histogram
    counts = computeHistogramChannel(img);

    % Hitung CDF (cumulative)
    cdf = cumsum(counts);

    % Normalisasi CDF
    cdfNormalized = (cdf / numPixels) * 255;
    cdfNormalized = round(cdfNormalized);

    % Transformasi/Mapping
    result = zeros(m, n);
    for i = 1:m
        for j = 1:n
            r = round(img(i, j)) + 1; % round() untuk keamanan
            r = max(1, min(256, r)); % Clamp
            result(i, j) = cdfNormalized(r);
        end
    end
end

```

Prosedur:

1. Hitung histogram 256 bins menggunakan fungsi custom
2. Hitung CDF kumulatif dengan `cumsum()`
3. Normalisasi CDF: bagi dengan jumlah piksel, kali 255
4. Bulatkan hasil CDF ke bilangan bulat
5. Untuk setiap piksel, cari nilai CDF yang sesuai dan petakan

b. Histogram Equalization Global (RGB)

```
function result = equalizeGlobal(img)
    [m, n, ~] = size(img);
    numPixels = m * n;

    result = zeros(size(img));

    % Equalize setiap channel
    for ch = 1:3
        channel = double(img(:, :, ch));

        % Hitung histogram channel
        counts = computeHistogramChannel(channel);

        % Hitung CDF
        cdf = cumsum(counts);

        % Normalisasi CDF
        cdfNormalized = round((cdf / numPixels) * 255);

        % Transformasi
        for i = 1:m
            for j = 1:n
                r = round(channel(i, j)) + 1;
                r = max(1, min(256, r));
                result(i, j, ch) = cdfNormalized(r);
            end
        end
    end
end
```

Perbedaan dengan Grayscale:

- Setiap channel (R, G, B) diproses secara independen
- Menggunakan loop for ch = 1:3 untuk ketiga channel
- Masing-masing channel memiliki histogram dan CDF sendiri

c. Histogram Equalization HSV

```
function result = equalizeHSV(img)
    % Konversi RGB ke HSV
    hsv = rgb2hsvCustom(img);

    % Ambil channel V (berada pada range [0, 1])
    v = double(hsv(:, :, 3));
    [m, n] = size(v);

    % Konversi V dari [0,1] ke [0,255]
    vScaled = v * 255;
```

```

% Equalize V channel
counts = computeHistogramChannel(vScaled);
cdf = cumsum(counts);
cdfNormalized = round((cdf / (m * n)) * 255);

resultV = zeros(m, n);
for i = 1:m
    for j = 1:n
        r = round(vScaled(i, j)) + 1;
        r = max(1, min(256, r));
        resultV(i, j) = cdfNormalized(r);
    end
end

% Konversi hasil dari [0,255] kembali ke [0,1]
resultV_normalized = resultV / 255;
resultV_normalized = max(0, min(1, resultV_normalized));
hsv(:, :, 3) = resultV_normalized;

% Konversi HSV ke RGB
result = hsv2rgbCustom(hsv);
end

```

Prosedur HSV:

1. Konversi RGB ke HSV
2. Ambil channel V (kecerahan)
3. Normalisasi V ke [0, 255]
4. Equalize hanya channel V
5. Normalisasi kembali ke [0, 1]
6. Konversi HSV ke RGB

Keuntungan HSV:

- Hanya mengubah kecerahan, tidak mengubah warna (hue dan saturation)
- Menghindari pergeseran warna yang mungkin terjadi pada mode global

d. Fungsi Konversi Color Space

Konversi RGB ke HSV menggunakan formula:

```

function hsv = rgb2hsvCustom(rgb)
    r = double(rgb(:, :, 1)) / 255;
    g = double(rgb(:, :, 2)) / 255;
    b = double(rgb(:, :, 3)) / 255;

    v = max(max(r, g), b);
    c = v - min(min(r, g), b);

    s = zeros(size(v));
    s(v ~= 0) = c(v ~= 0) ./ v(v ~= 0);
    % ... (perhitungan hue)

```



```

        hsv = cat(3, h, s, v);
    end

```

Konversi RGB ke YCbCr menggunakan standar ITU-R BT.601:

```

function ycbcr = rgb2ycbcrCustom(rgb)
    r = double(rgb(:, :, 1));
    g = double(rgb(:, :, 2));
    b = double(rgb(:, :, 3));

    ycbcr(:, :, 1) = 0.299 * r + 0.587 * g + 0.114 * b;      % Y
    ycbcr(:, :, 2) = 128 - 0.168736 * r - 0.331264 * g + 0.5 * b; %
Cb
    ycbcr(:, :, 3) = 128 + 0.5 * r - 0.418688 * g - 0.081312 * b; %
Cr
end

```

3.6 Implementasi Histogram Matching

Histogram Matching digunakan untuk mengubah distribusi intensitas citra input agar mendekati distribusi intensitas citra referensi. Proses dilakukan dengan menghitung histogram dan CDF dari citra input dan citra referensi, kemudian membuat Look-Up Table untuk memetakan setiap nilai intensitas input ke nilai intensitas pada citra referensi.

Fungsi utama : histogramMatching

```

function [outputImg, info] = histogramMatching(img, refImg, varargin)

    if nargin < 2 || isempty(img) || isempty(refImg)
        error('Citra input dan citra referensi diperlukan.');
```

```

    end

    p = inputParser;
    addRequired(p, 'img', @isnumeric);
    addRequired(p, 'refImg', @isnumeric);
    addParameter(p, 'mode', 'global', @ischar);
    parse(p, img, refImg, varargin{:});

    mode = lower(p.Results.mode);
    isRGB = (ndims(img) == 3 && size(img, 3) == 3);

    if ~isRGB
        [mapping, cdfIn, cdfRef] = ...
            computeHistogramMatchingLUT(img, refImg);

        outputImg = applyHistogramMapping(img, mapping);
        currentMode = 'grayscale';
    else
        switch mode
            case 'global'

```

```

        [mapping, cdfIn, cdfRef] = ...
            computeHistogramMatchingLUT(img, refImg);

        outputImg = applyHistogramMapping( ...
            img, mapping, 'global');

        currentMode = 'global';

    case 'hsv'
        hsvIn = rgb2hsvLocal(img);
        hsvRef = rgb2hsvLocal(refImg);

        vIn = hsvIn(:, :, 3) * 255;
        vRef = hsvRef(:, :, 3) * 255;

        [mapping, cdfIn, cdfRef] = ...
            computeHistogramMatchingLUT(vIn, vRef);

        outputImg = applyHistogramMapping( ...
            img, mapping, 'hsv');

        currentMode = 'hsv';

    case 'ycbcr'
        ybcrIn = rgb2ycbcrLocal(img);
        ybcrRef = rgb2ycbcrLocal(refImg);

        yIn = ybcrIn(:, :, 1);
        yRef = ybcrRef(:, :, 1);

        [mapping, cdfIn, cdfRef] = ...
            computeHistogramMatchingLUT(yIn, yRef);

        outputImg = applyHistogramMapping( ...
            img, mapping, 'ycbcr');

        currentMode = 'ycbcr';

    end
end
end
end

```

Fungsi tersebut menerima citra input dan citra referensi. Untuk citra grayscale, proses dilakukan secara langsung. Untuk citra RGB, tersedia tiga mode, yaitu global, HSV, dan YCbCr. Pada mode HSV, proses matching dilakukan pada channel Value, sedangkan pada mode YCbCr dilakukan pada channel Y.

a. Pembentukan Look-Up Table

Proses pembentukan mapping dilakukan menggunakan fungsi `computeHistogramMatchingLUT`. Histogram input dan referensi dihitung menggunakan

fungsi histogram yang telah dibuat sebelumnya. Setelah itu, CDF dari kedua histogram dihitung dan dinormalisasi ke rentang 0 sampai 1.

```
function cdf = calculateNormalizedCDF(counts)

    counts = double(counts(:));
    cdf = zeros(256, 1);
    cumulative = 0;

    for i = 1:256
        cumulative = cumulative + counts(i);
        cdf(i) = cumulative;
    end

    total = cdf(256);

    if total > 0
        cdf = cdf / total;
    else
        cdf = zeros(256, 1);
    end
end
```

Selanjutnya, setiap nilai CDF pada citra input dicari nilai CDF referensi yang paling dekat. Nilai intensitas yang ditemukan kemudian digunakan sebagai hasil mapping. Proses ini menghasilkan LUT dengan 256 nilai intensitas.

```
function mapping = findClosestMapping(cdfIn, cdfRef)

    mapping = zeros(256, 1, 'uint8');

    for r = 0:255
        r_idx = r + 1;
        fr = cdfIn(r_idx);

        minDiff = Inf;
        bestZ = 0;

        for z = 0:255
            z_idx = z + 1;
            diff = abs(cdfRef(z_idx) - fr);

            if diff < minDiff
                minDiff = diff;
                bestZ = z;
            end
        end
    end
```

```

        mapping(r_idx) = uint8(bestZ);
    end
end

```

LUT yang telah diperoleh kemudian diterapkan pada setiap piksel citra menggunakan fungsi `applyHistogramMapping`. Nilai intensitas piksel digunakan sebagai index untuk mengambil nilai baru dari LUT.

```

function out = applyMappingChannel(channel, lut)

    ch = double(channel);
    idx = round(ch) + 1;
    idx = max(1, min(256, idx));

    lut = uint8(lut(:));
    out = lut(idx);
end

```

Dengan demikian, proses Histogram Matching terdiri dari tiga tahap utama, yaitu menghitung CDF citra input dan referensi, membentuk LUT berdasarkan nilai CDF yang paling dekat, kemudian menerapkan LUT pada citra input. Hasil akhirnya adalah citra dengan distribusi intensitas yang mendekati distribusi citra referensi.

3.7 Implementasi Image Filtering

Image Filtering digunakan untuk memperbaiki kualitas citra dengan memanfaatkan nilai piksel di sekitar setiap piksel. Pada implementasi ini digunakan dua jenis filtering, yaitu Linear Filtering dan Median Filtering.

a. Linear Filtering

Linear Filtering dilakukan menggunakan kernel atau mask. Setiap nilai piksel pada neighborhood dikalikan dengan nilai kernel yang sesuai, kemudian dijumlahkan untuk menghasilkan nilai piksel baru.

```

region = paddedImage( ...
    i:i + kernelHeight - 1, ...
    j:j + kernelWidth - 1);

output(i,j) = sum(sum(region .* kernel));

```

Beberapa kernel dapat digunakan sesuai kebutuhan, seperti Mean untuk smoothing, Gaussian untuk mengurangi noise, serta Sharpen dan Sobel untuk meningkatkan detail atau mendeteksi tepi.

b. Median Filtering

Median Filtering merupakan filtering non-linear yang mengganti nilai piksel dengan nilai median dari neighborhood di sekitarnya. Metode ini terutama digunakan untuk mengurangi noise seperti salt and pepper.

```
values = region(:);  
values = sort(values);  
  
middle = ceil(length(values) / 2);  
output(i,j) = values(middle);
```

Ukuran window yang digunakan berupa ukuran ganjil seperti 3×3 , 5×5 , dan 7×7 . Ukuran window yang lebih besar menghasilkan efek smoothing yang lebih kuat, tetapi dapat mengurangi detail citra.

Kedua metode kemudian diintegrasikan ke dalam GUI sehingga pengguna dapat memilih jenis filtering dan parameternya. Hasil filtering ditampilkan pada Output Image dan Output Histogram.

BAB IV

PENGUJIAN DAN ANALISIS

4.1 Skenario Pengujian

Pengujian dilakukan untuk mengevaluasi kemampuan program dalam menganalisis histogram citra dan meningkatkan kualitas citra menggunakan metode *image enhancement* yang telah diimplementasikan. Pengujian dilakukan terhadap seluruh citra pada setiap *subfolder* dataset resmi.

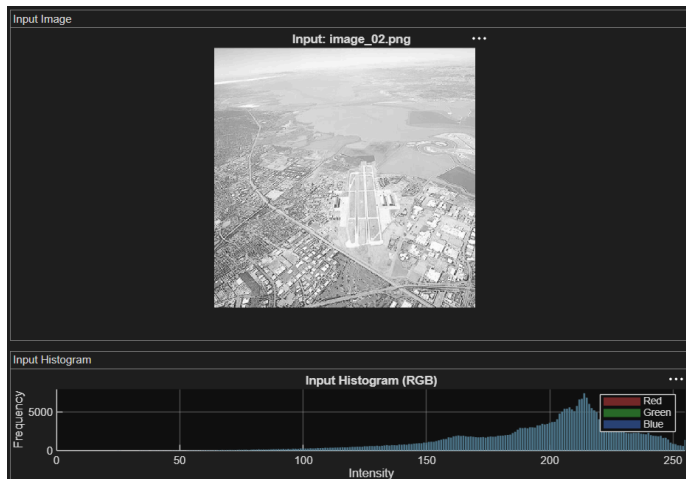
Pada setiap kasus, analisis mencakup identifikasi permasalahan kualitas visual citra, metode yang digunakan, parameter yang diterapkan, perubahan citra dan histogram, serta keterbatasan atau artefak yang muncul. Dengan demikian, pengujian tidak hanya memeriksa apakah program dapat menghasilkan citra keluaran, tetapi juga mengevaluasi kesesuaian metode *enhancement* terhadap karakteristik masing-masing citra.

4.2 Pengujian Histogram Citra

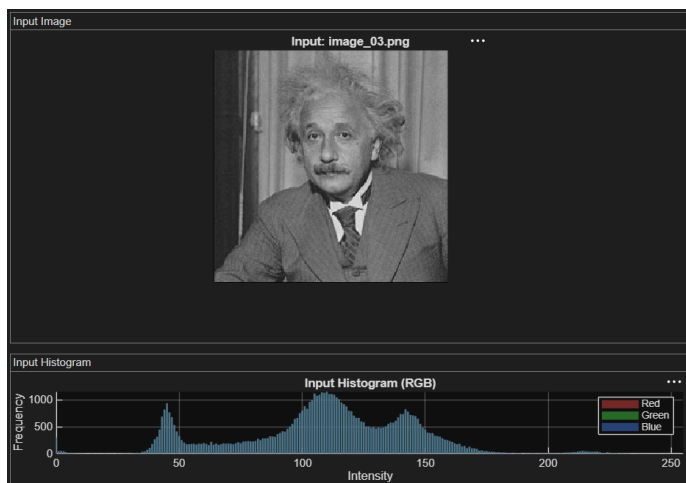
a. image_01.png



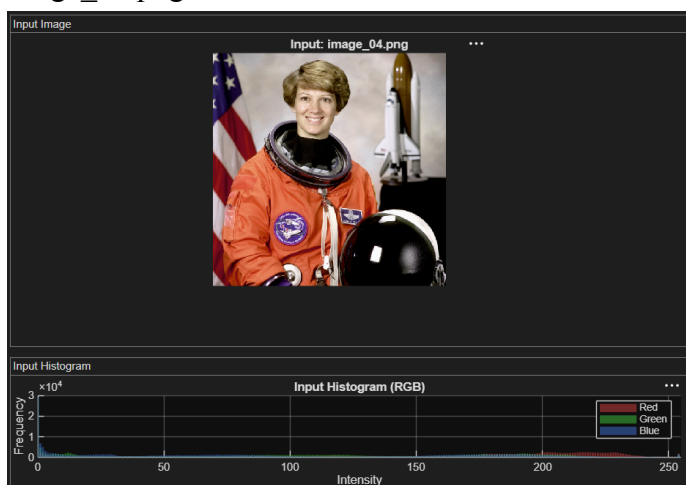
b. image_02.png



c. image_03.png

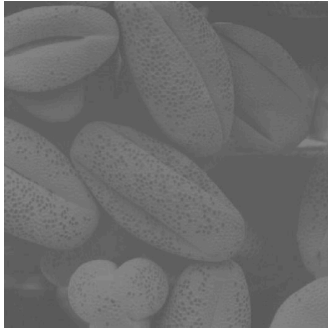


d. image_04.png

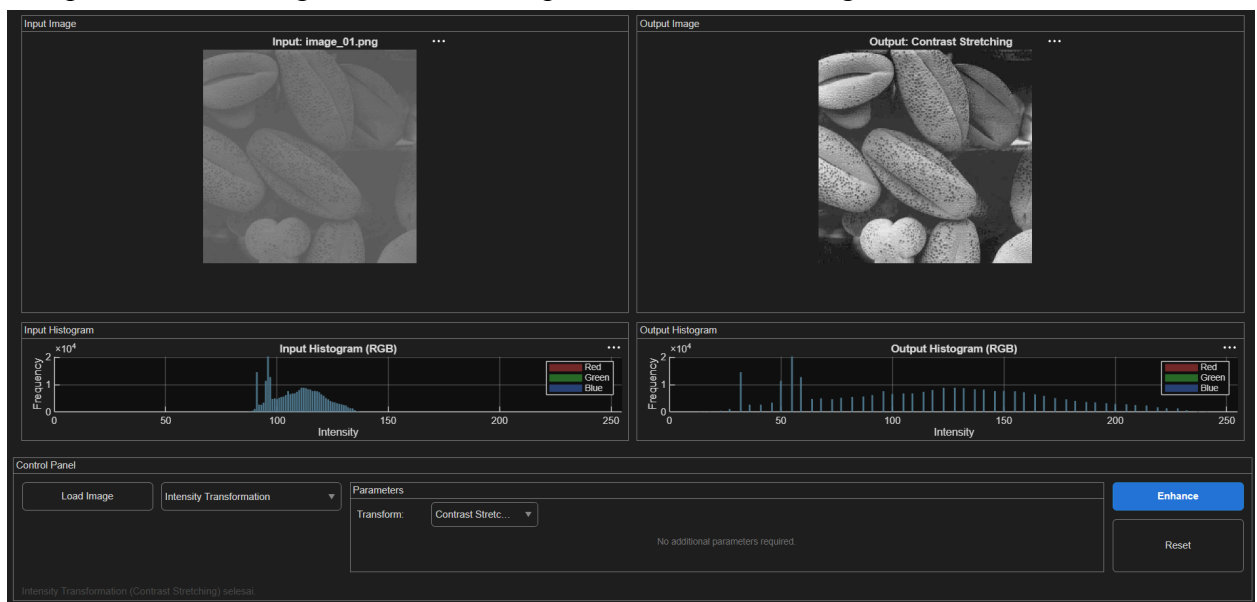


4.3 Pengujian Kasus 1

a. image_01.png



Citra *Seeds* di atas memiliki kontras yang sangat rendah dan didominasi oleh intensitas abu-abu gelap sehingga detail tekstur pada permukaan citra kurang terlihat jelas. Berdasarkan histogram, distribusi intensitas piksel terkonsentrasi pada rentang yang sempit di area *mid-tone*. Oleh karena itu, digunakan metode *Contrast Stretching* untuk memperluas rentang intensitas piksel ke rentang dinamis 0–255.

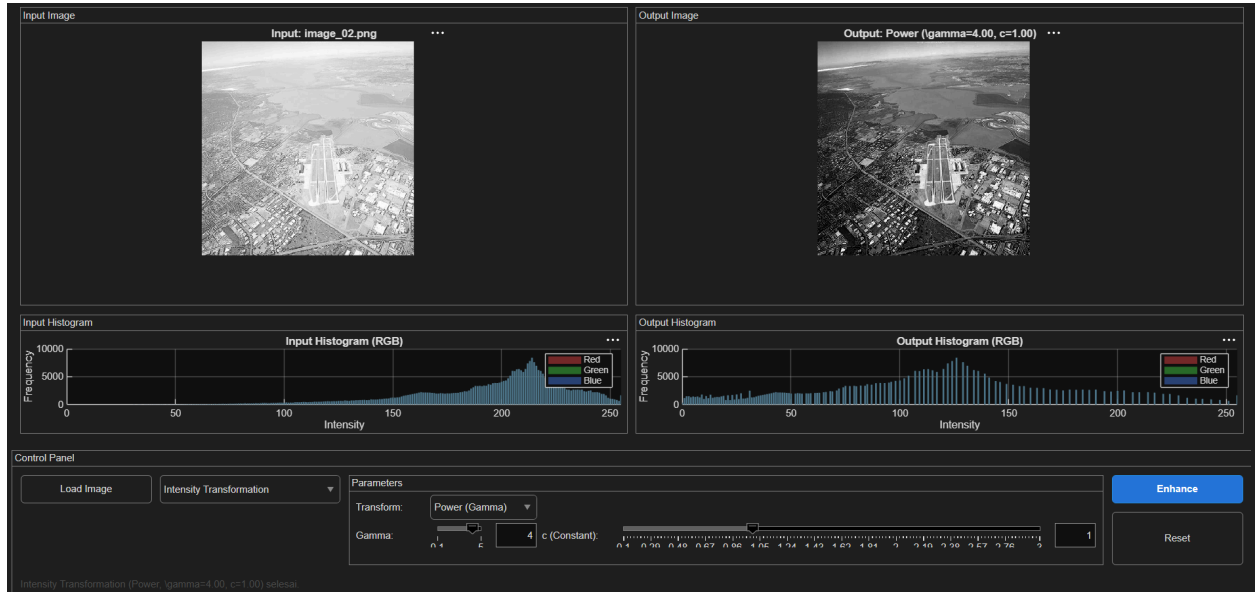


Berdasarkan hasil di atas, dapat dilihat bahwa *Contrast Stretching* meningkatkan perbedaan intensitas antarpiksel sehingga detail tekstur pada permukaan citra menjadi lebih terlihat. Dapat dilihat juga bahwa histogram output sudah terdistribusi merata ke seluruh rentang intensitas piksel.

b. image_02.png



Pada citra di atas, terlihat wilayah pesisir yang terlalu terang atau pucat (*washed out/overexposed*) pada area atas dan memiliki nilai intensitas abu-abu yang tinggi. Berdasarkan kondisi tersebut, digunakan metode *Gamma Correction* dengan nilai $\gamma > 1$. Nilai gamma yang lebih besar dari 1 digunakan untuk menurunkan intensitas pada area yang terang sehingga wilayah yang sebelumnya terlalu terang menjadi lebih gelap dan detail pada area tersebut dapat terlihat lebih jelas.

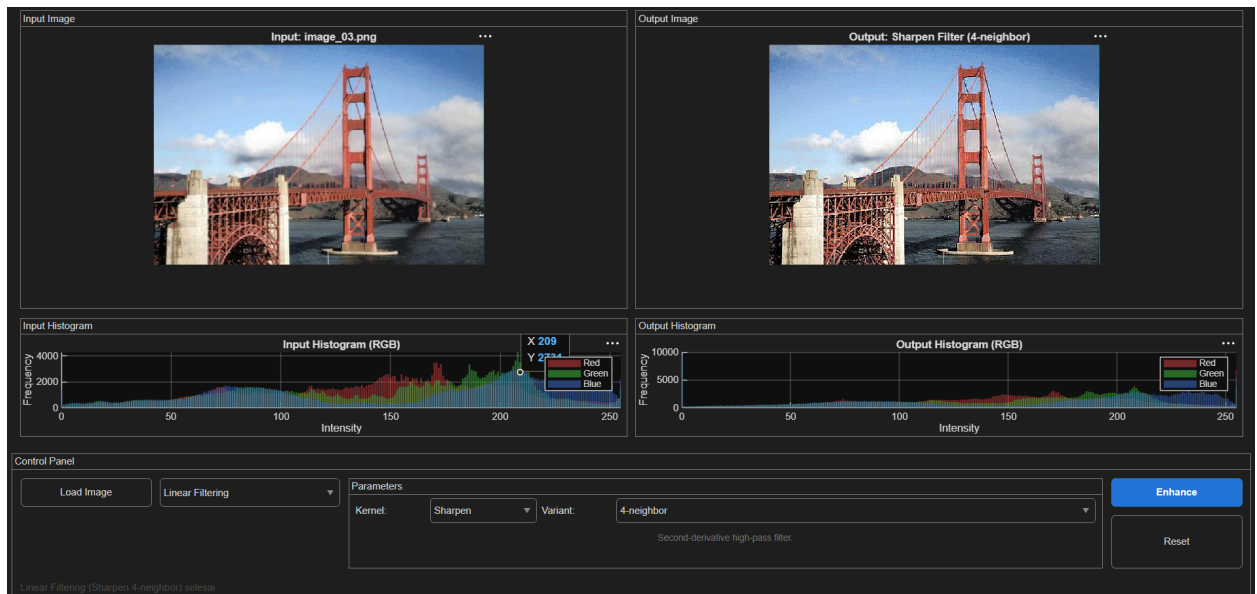


Berdasarkan hasil di atas, dapat dilihat bahwa *Gamma Correction* berhasil menurunkan intensitas pada area citra yang terlalu terang. Bagian atas gambar yang sebelumnya terlihat pucat menjadi lebih gelap sehingga detail pada area tersebut lebih terlihat. Histogram hasil juga menunjukkan pergeseran distribusi intensitas ke arah nilai yang lebih rendah. Nilai $\gamma = 4$ dipilih karena menghasilkan penurunan intensitas yang cukup signifikan pada area terang, tetapi tetap mempertahankan detail objek pada citra tanpa menyebabkan keseluruhan citra menjadi terlalu gelap. Adapun nilai $c = 1$ dipilih karena tidak diperlukan tambahan penguatan atau pelemahan intensitas melalui nilai c .

c. image_03.png

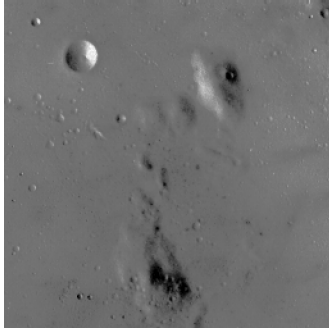


Citra jembatan di atas tampak sedikit buram pada bagian tepi (*edges*) dan detail halus, seperti kabel pilar jembatan, struktur baja, serta tekstur awan/pegunungan di latar belakang. Perbedaan intensitas antarpiksel tetangga pada area batas objek (*boundary*) tidak begitu tinggi sehingga persepsi ketajaman visual berkurang. Penerapan filter penajaman dengan linear filtering akan memperjelas detail-detail gambar.

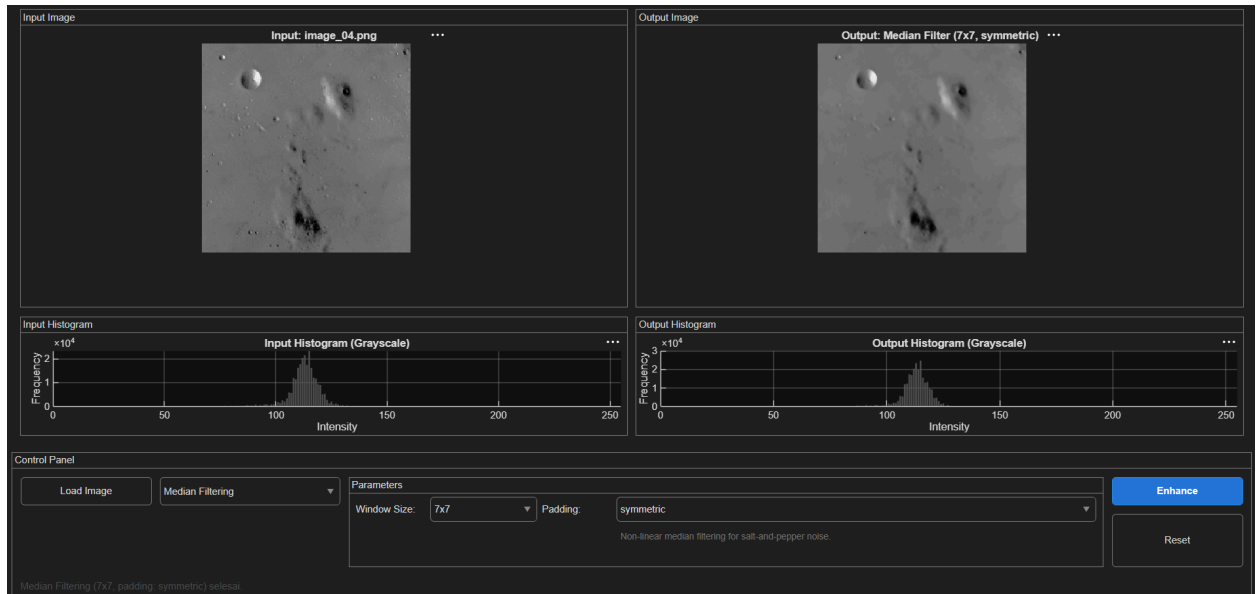


Filter *sharpening* linear dipilih karena filter ini mengekstraksi turunan kedua dari citra untuk memperkuat komponen frekuensi tinggi. Metode ini memperbesar selisih intensitas pada batas antar objek tanpa mengaburkan detail. Efeknya, kabel pilar jembatan dan kerangka rangka struktur menjadi tampak jauh lebih jelas dan tajam secara visual dibanding citra aslinya. Dapat dilihat juga pada histogram bahwa proses *sharpening* menaikkan nilai piksel terang di daerah tepi dan menurunkan nilai piksel gelap di sekitarnya. Hal ini menyebabkan distribusi frekuensi intensitas pada histogram menjadi lebih menyebar (*stretched*) ke arah batas ekstrem (mendekati 0 dan 255). Nilai frekuensi relatif melandai dan terdistribusi lebih merata di rentang intensitas yang lebih luas karena variasi lokal antar piksel tetangga meningkat.

d. image_04.png



Citra permukaan luar ruang/astronomi di atas memiliki bintik-bintik *noise* yang tersebar pada beberapa bagian citra dan menyerupai *salt-and-pepper noise*. *Noise* tersebut menyebabkan munculnya piksel dengan intensitas yang jauh berbeda dari piksel di sekitarnya. Oleh karena itu, digunakan metode *Median Filtering* untuk mengurangi *noise* tersebut. Adapun *Median filter* dipilih dibandingkan *Mean Filtering* karena nilai median lebih tahan terhadap piksel dengan intensitas ekstrem.



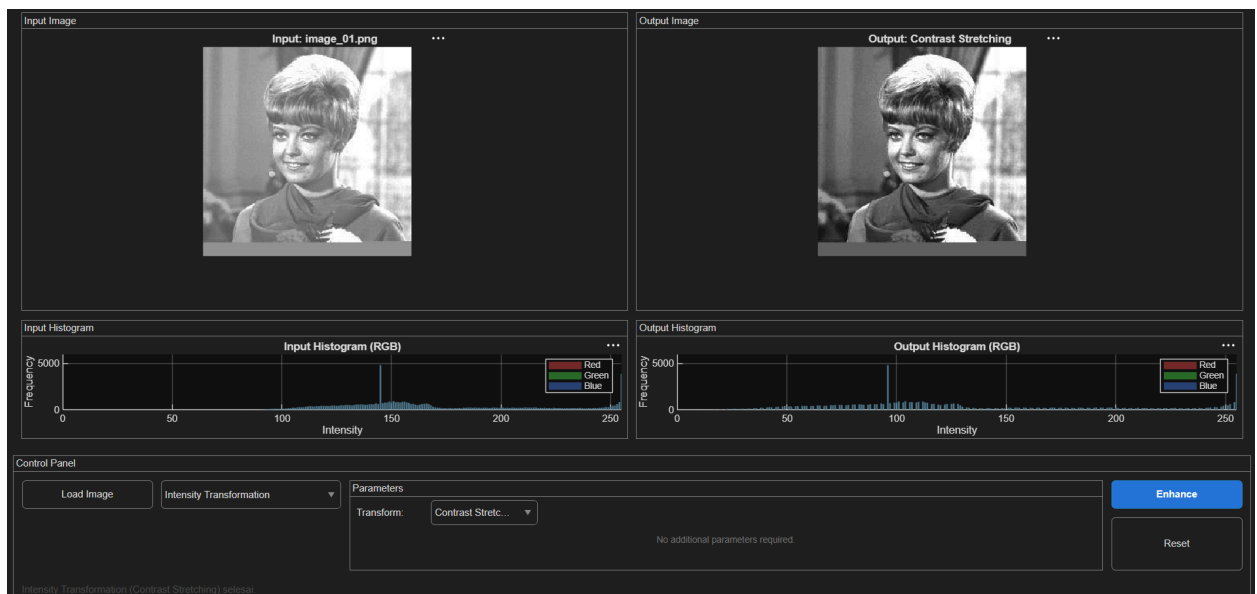
Berdasarkan hasil *enhancement*, bintik-bintik *noise* pada citra terlihat berkurang dan permukaan citra menjadi lebih halus. Detail utama objek masih dapat dipertahankan, sementara piksel-piksel dengan intensitas ekstrem akibat *noise* tidak lagi terlalu menonjol. Ukuran *kernel* 7×7 dipilih karena distribusi *noise* pada citra cukup banyak dan sangat tersebar sehingga diperlukan area *neighborhood* yang lebih luas untuk memperoleh nilai median yang dapat merepresentasikan intensitas piksel di sekitarnya. *Symmetric padding* juga dipilih karena memantulkan nilai piksel di sekitar tepi sehingga distribusi intensitas pada area batas tetap lebih alami. Penggunaan *symmetric padding* juga menghindari penambahan piksel bernilai nol seperti pada *zero padding* yang dapat menghasilkan artefak gelap pada bagian tepi citra.

4.4 Pengujian Kasus 2

a. image_01.png



Citra *Zelda* di atas memiliki kontras yang cukup rendah dan didominasi oleh intensitas abu-abu terang sehingga detail tekstur seperti pada detail pakaian dan rambut kurang terlihat jelas. Pantulan cahaya dari jendela menyebabkan beberapa bagian citra terlalu terang. Berdasarkan histogram, distribusi intensitas piksel terkonsentrasi pada rentang yang cukup terbatas. Oleh karena itu, digunakan metode *Contrast Stretching* untuk memperluas rentang intensitas piksel ke rentang dinamis 0–255.

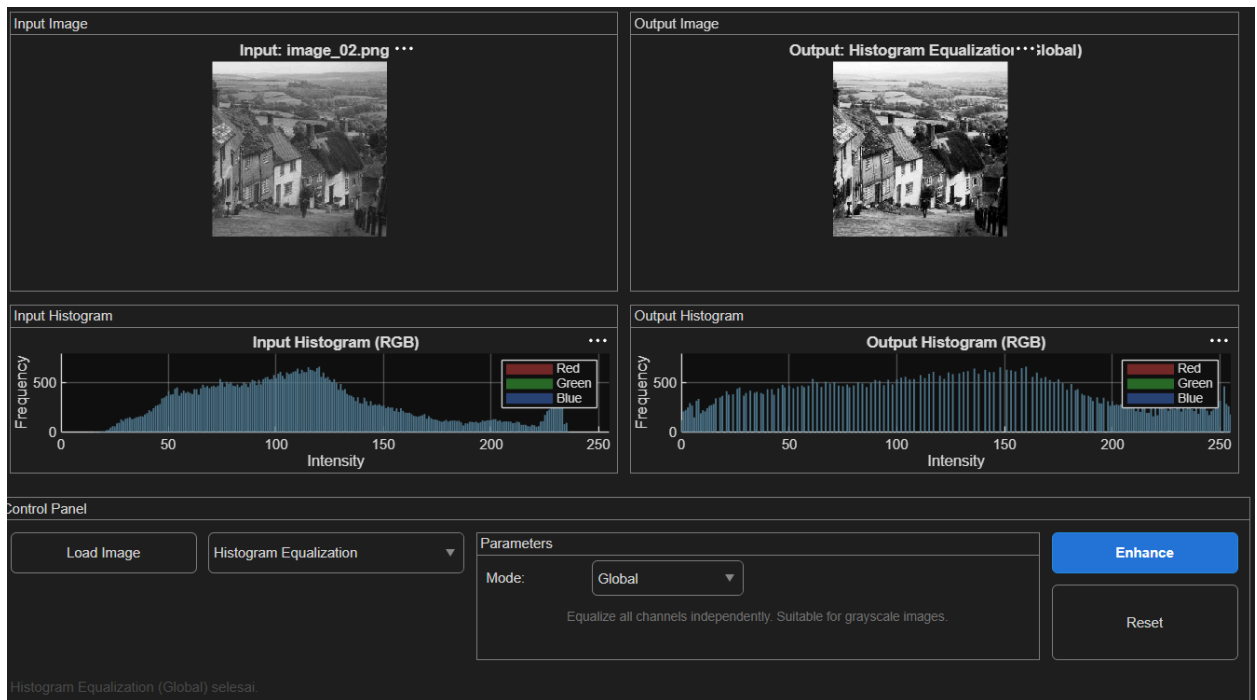


Berdasarkan hasil di atas, dapat dilihat bahwa *Contrast Stretching* meningkatkan perbedaan intensitas antarpiksel sehingga detail tekstur pada pakaian dan struktur rambut menjadi lebih terlihat. Area yang sebelumnya didominasi oleh intensitas abu-abu terang juga memiliki variasi intensitas yang lebih jelas setelah proses enhancement. Pada histogram output, distribusi intensitas menjadi lebih tersebar dibandingkan histogram input karena rentang intensitas piksel diperluas ke rentang yang lebih besar. Dengan demikian, kontras citra meningkat dan detail pada objek menjadi lebih mudah dibedakan.

b. image_02.png



Citra perumahan di atas, terlihat grayscale secara visual, detail masih terlihat cukup jelas, kontrasnya terlihat agak rendah, karena banyak piksel berada di intensitas menengah dan tidak terlihat noise yang dominan. Namun walaupun gambar terlihat grayscale, citra ini terdiri dari 3 channel (RGB), bukan satu grayscale histogram. Namun karena gambar terlihat grayscale, kemungkinan nilai R, G, dan B mirip sehingga ketiga histogramnya terlihat hampir bertumpuk. Jadi untuk gambar ini, kita bisa menggunakan histogram Equalization. Histogram citra cenderung terkonsentrasi pada intensitas menengah sehingga kontras citra masih terbatas. Histogram Equalization digunakan untuk menyebarkan distribusi intensitas agar detail pada area gelap dan terang lebih terlihat.

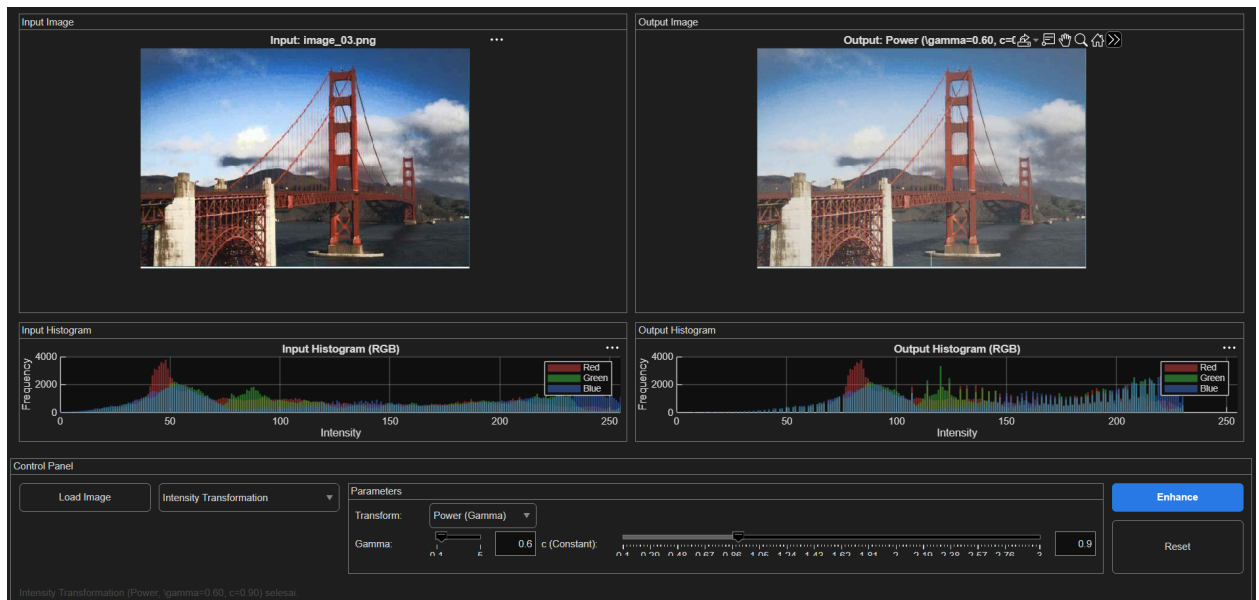


Setelah enhancement, kontras citra meningkat dan detail pada area gelap maupun terang menjadi lebih terlihat. Histogram output juga menunjukkan distribusi intensitas yang lebih tersebar dibandingkan histogram input.

c. image_03.png



Pada citra jembatan di atas, terlihat bagian atas citra, terutama pada area yang berbentuk melengkung, memiliki intensitas yang cukup rendah sehingga detail pada area tersebut kurang terlihat jelas. Berdasarkan kondisi tersebut, digunakan metode *Gamma Correction* dengan nilai $\gamma < 1$. Nilai gamma yang lebih kecil dari 1 digunakan untuk meningkatkan intensitas pada area yang gelap sehingga bagian yang sebelumnya terlalu gelap menjadi lebih terang dan detail pada area tersebut.

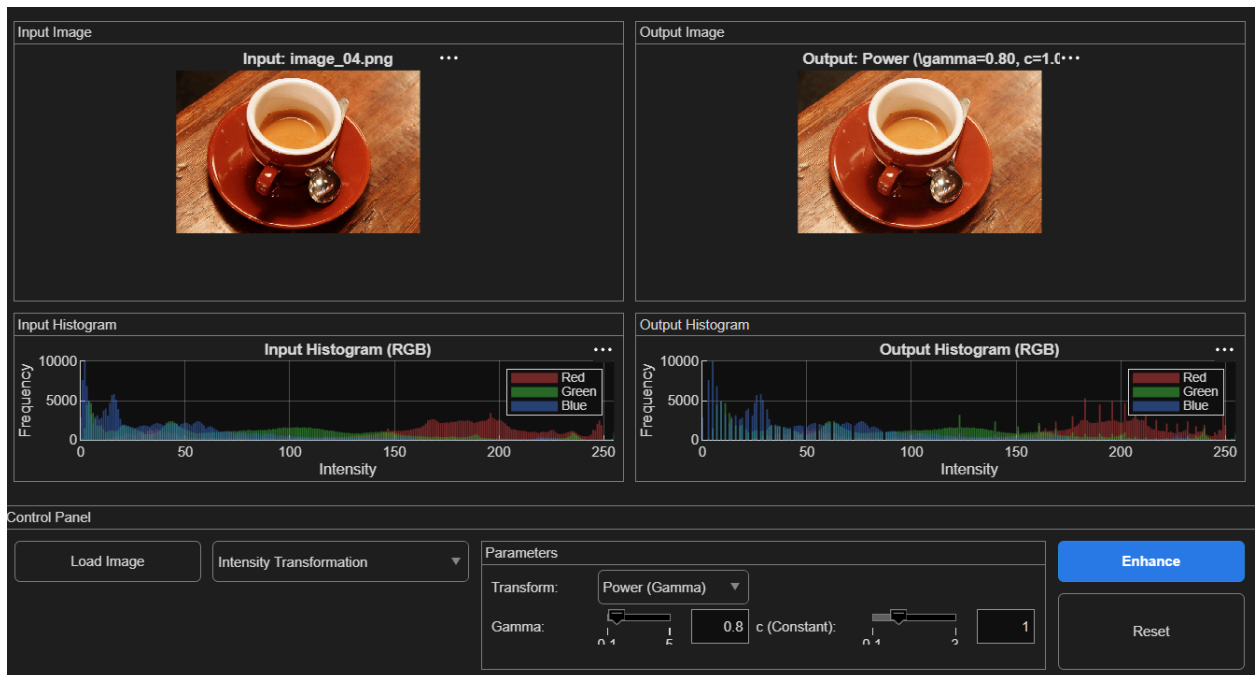


Berdasarkan hasil di atas, dapat dilihat bahwa *Gamma Correction* berhasil meningkatkan intensitas pada area citra yang terlalu gelap. Bagian atas gambar yang sebelumnya terlihat gelap menjadi lebih terang sehingga area tersebut lebih terlihat. Histogram hasil juga menunjukkan pergeseran distribusi intensitas ke arah nilai yang lebih tinggi. Nilai $\gamma = 0.6$ dipilih karena memberikan peningkatan intensitas yang cukup pada area gelap, tetapi tetap mempertahankan detail pada bagian citra lainnya. Adapun nilai $c = 0.9$ dipilih untuk mengurangi sedikit penguatan intensitas dari gamma correction sehingga hasil akhir tidak menjadi terlalu terang.

d. image_04.png



Citra merupakan citra RGB dengan distribusi intensitas yang berbeda pada setiap channel. Channel Red cenderung memiliki intensitas lebih tinggi, sedangkan Blue lebih banyak berada pada intensitas rendah. Hal ini sesuai dengan dominasi warna merah dan coklat pada citra. Oleh karena itu, Power Transformation dengan Nilai $\gamma = 0.8$ digunakan. Gamma kurang dari 1 digunakan untuk meningkatkan intensitas pada area yang relatif gelap sehingga detail pada area tersebut dapat lebih terlihat.



Hasil enhancement dengan gamma 0.8 membuat citra sedikit lebih terang, terutama pada area gelap. Perubahan masih relatif natural dan detail citra tetap terlihat dengan baik. Distribusi intensitas bergeser ke arah intensitas yang lebih tinggi. Hal ini menunjukkan bahwa area gelap pada citra mengalami peningkatan intensitas. Secara visual, citra menjadi sedikit lebih terang dengan detail pada area gelap yang lebih terlihat, sementara karakteristik warna citra masih dipertahankan.

4.5 Pengujian Kasus 3

a. image_01.png

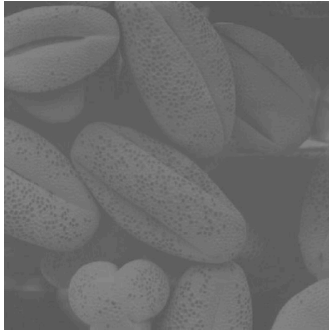


Citra perbukitan di atas memiliki kontras yang rendah sehingga perbedaan intensitas antara bagian rumput dengan pepohonan kurang terlihat jelas. Berdasarkan histogram, distribusi intensitas piksel terkonsentrasi pada area *mid-tone* dengan rentang yang relatif sempit. Oleh karena itu, digunakan metode *Contrast Stretching* untuk memperluas rentang intensitas piksel sehingga perbedaan intensitas antarbagiannya menjadi lebih terlihat.

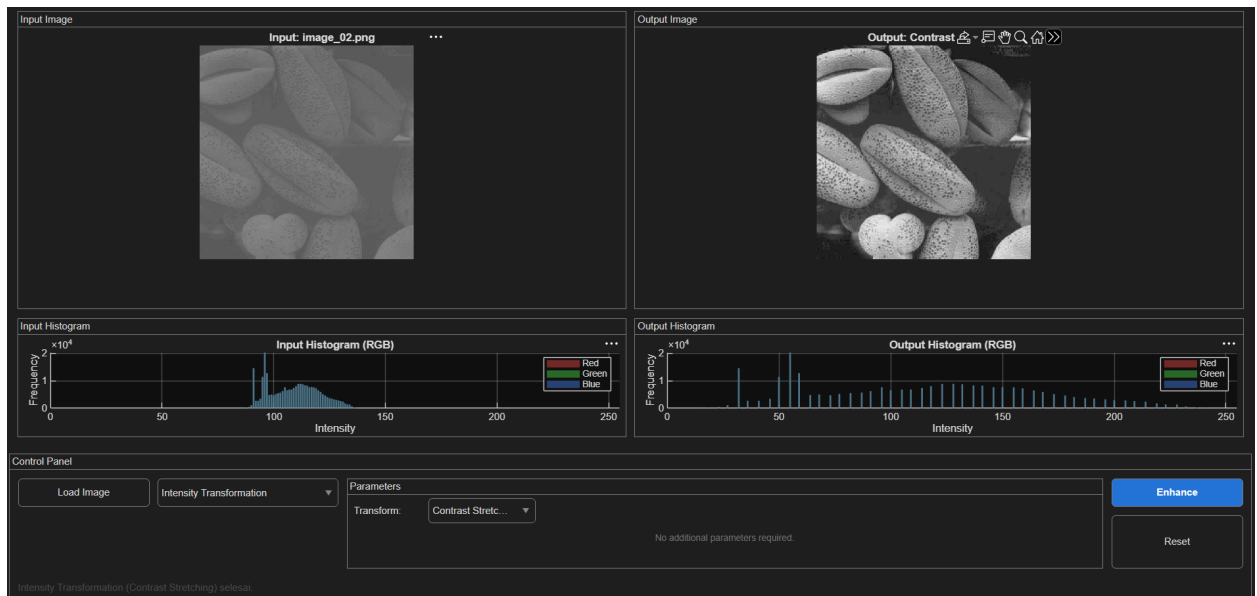


Berdasarkan hasil di atas, dapat dilihat bahwa *Contrast Stretching* meningkatkan perbedaan intensitas antarpiksel sehingga detail pada perbukitan menjadi lebih terlihat. Pada histogram output, distribusi intensitas menjadi lebih tersebar dibandingkan histogram input karena rentang intensitas piksel diperluas.

b. image_02.png

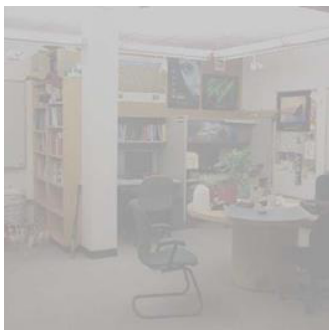


Citra *Seeds* di atas memiliki kontras yang sangat rendah dan didominasi oleh intensitas abu-abu gelap sehingga detail tekstur pada permukaan citra kurang terlihat jelas. Berdasarkan histogram, distribusi intensitas piksel terkonsentrasi pada rentang yang sempit di area *mid-tone*. Oleh karena itu, digunakan metode *Contrast Stretching* untuk memperluas rentang intensitas piksel ke rentang dinamis 0–255.

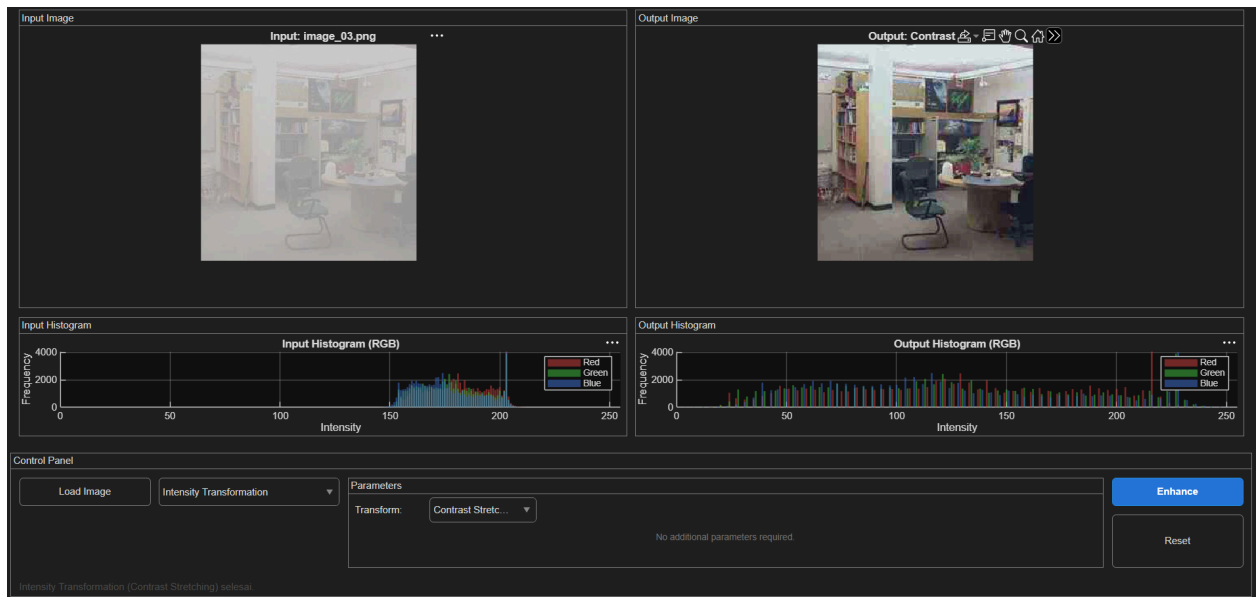


Berdasarkan hasil di atas, dapat dilihat bahwa *Contrast Stretching* meningkatkan perbedaan intensitas antarpiksel sehingga detail tekstur pada permukaan citra menjadi lebih terlihat. Dapat dilihat juga bahwa histogram output sudah terdistribusi merata ke seluruh rentang intensitas piksel.

c. image_03.png

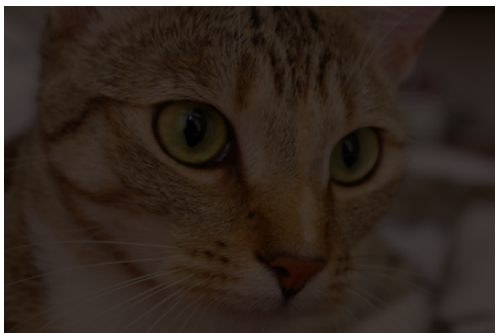


Citra interior ruangan kantor di atas memiliki tingkat kecerahan yang sangat tinggi sehingga sebagian besar area citra terlihat pudar dan detail barang-barang seperti kursi, rak buku, dan poster di dinding, kurang terlihat jelas. Berdasarkan histogram, distribusi intensitas piksel terkonsentrasi pada area intensitas tinggi yang mendekati 255. Oleh karena itu, digunakan metode *Contrast Stretching* untuk memperluas rentang intensitas piksel sehingga perbedaan intensitas pada area citra yang terlalu terang menjadi lebih terlihat.



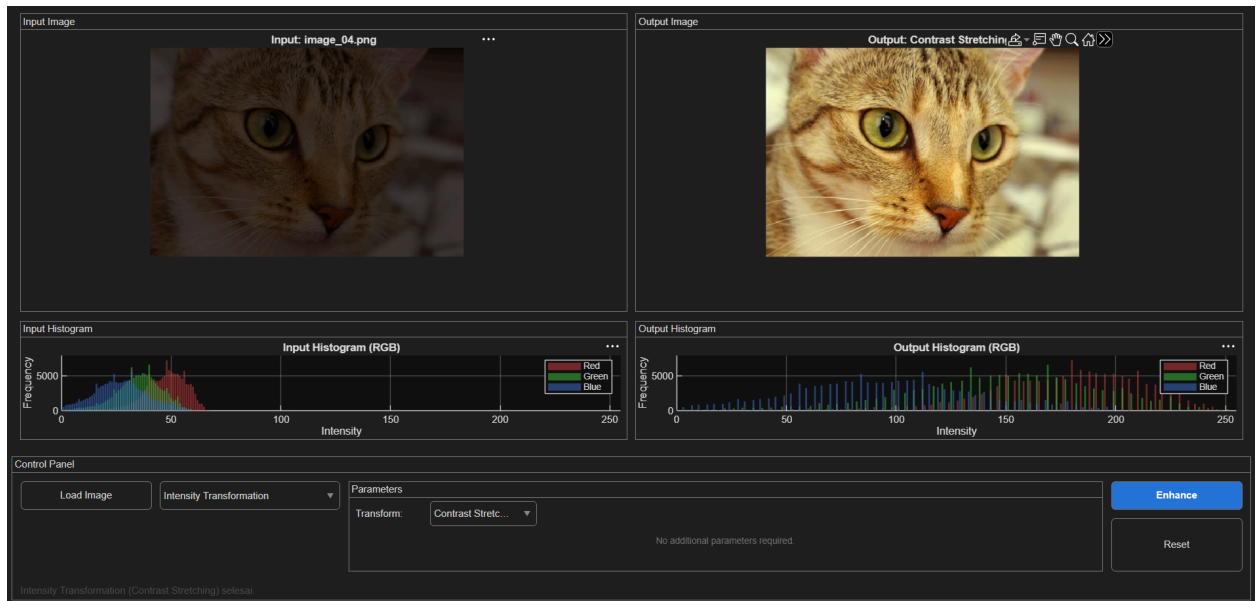
Berdasarkan hasil di atas, dapat dilihat bahwa *Contrast Stretching* meningkatkan perbedaan intensitas antarpiksel sehingga detail yang sebelumnya kurang terlihat menjadi lebih jelas. Area yang sebelumnya didominasi oleh warna putih juga menunjukkan variasi intensitas yang lebih besar setelah proses enhancement. Pada histogram output, distribusi intensitas menjadi lebih tersebar karena rentang intensitas piksel diperluas. Dengan demikian, kontras citra meningkat dan detail pada objek di dalam ruangan menjadi lebih mudah dibedakan.

d. image_04.png



Citra kucing di atas memiliki tingkat kecerahan yang rendah sehingga sebagian besar area citra terlihat gelap dan detail pada bagian tubuh serta wajah kucing kurang terlihat jelas. Berdasarkan histogram, distribusi intensitas piksel terkonsentrasi pada rentang intensitas

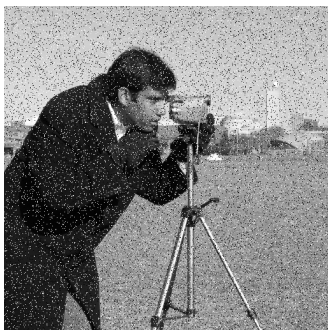
yang rendah dan relatif sempit. Oleh karena itu, digunakan metode *Contrast Stretching* untuk memperluas rentang intensitas piksel sehingga perbedaan intensitas pada area gelap menjadi lebih terlihat.



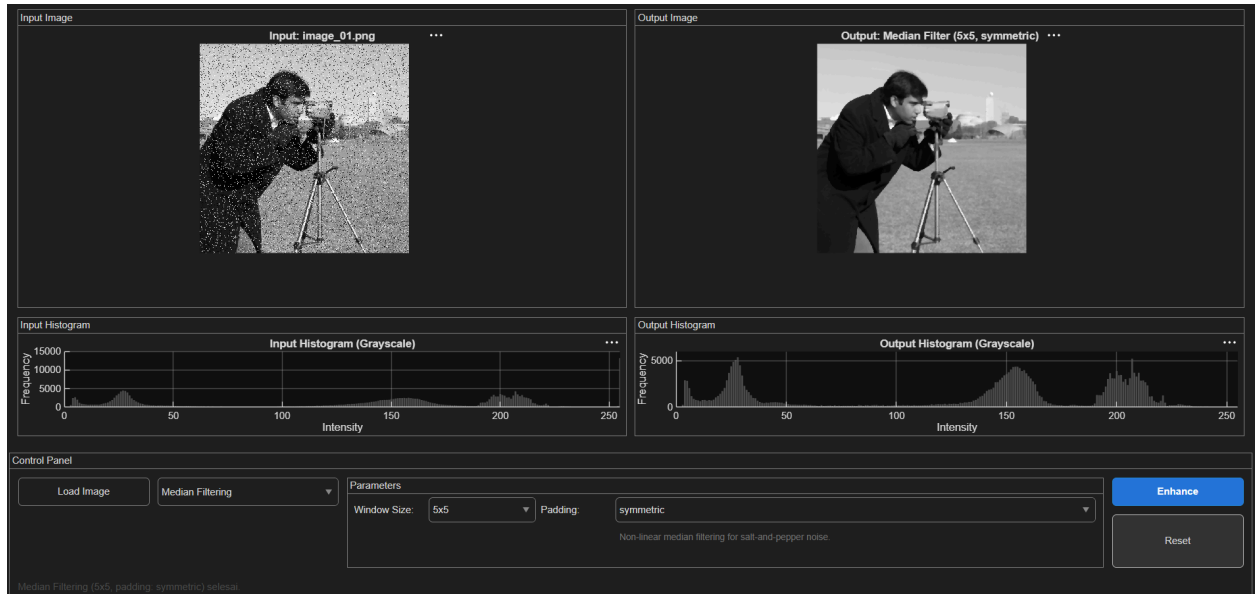
Berdasarkan hasil di atas, dapat dilihat bahwa *Contrast Stretching* meningkatkan perbedaan intensitas antarpiksel sehingga detail pada tubuh dan wajah kucing yang sebelumnya kurang terlihat menjadi lebih jelas. Pada histogram output, distribusi intensitas menjadi lebih tersebar dibandingkan histogram input karena rentang intensitas piksel diperluas. Dengan demikian, kontras citra meningkat dan detail pada area yang sebelumnya gelap menjadi lebih mudah dibedakan.

4.6 Pengujian Kasus 4

a. image_01.png

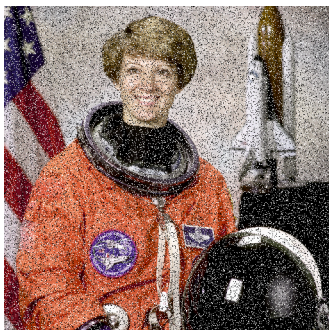


Citra *Camera* di atas memiliki *noise* berupa *salt-and-pepper noise*. *Noise* tersebut menyebabkan munculnya piksel dengan intensitas yang jauh berbeda dari piksel di sekitarnya. Oleh karena itu, digunakan metode *Median Filtering* untuk mengurangi *noise* tersebut. *Median filter* dipilih karena nilai median tahan terhadap piksel dengan intensitas ekstrem (*outlier*). Dengan demikian, piksel *noise* dapat ditekan tanpa terlalu mengaburkan tepi dan detail objek pada citra.

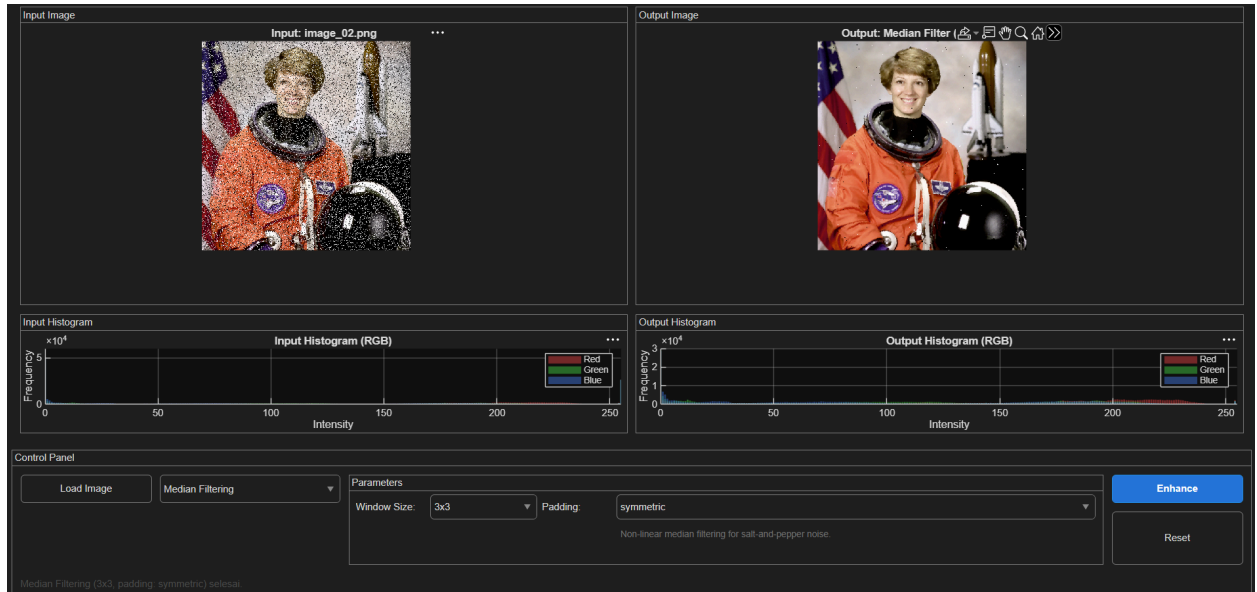


Berdasarkan hasil di atas, dapat dilihat bahwa *Median Filtering* berhasil mengurangi *noise* sehingga bintik-bintik pada citra menjadi lebih sedikit dan citra terlihat lebih halus. Pada histogram, rentang penyebaran intensitas piksel secara keseluruhan relatif tetap, tetapi batang histogram menjadi lebih tinggi pada beberapa nilai intensitas. Hal ini menunjukkan bahwa piksel-piksel ekstrem yang sebelumnya tersebar akibat *noise* telah digantikan oleh nilai median yang lebih merepresentasikan intensitas piksel di sekitarnya sehingga frekuensi pada intensitas tertentu meningkat. Ukuran kernel yang dipilih adalah 5×5 karena memberikan keseimbangan antara kemampuan mengurangi *noise* dan mempertahankan detail citra. Jika digunakan kernel 3×3 , area piksel yang diproses lebih kecil sehingga pengurangan sebagian *noise* masih dapat terlihat, terutama pada bagian perairan yang memiliki intensitas piksel yang relatif mirip dengan *noise* yang ada. Sementara kernel 7×7 memproses area yang lebih luas sehingga pengurangan *noise* menjadi lebih kuat, tetapi membuat detail halus dan tepi objek menjadi lebih buram. Adapun jenis *padding* yang digunakan adalah *symmetric padding* karena nilai piksel di sekitar batas citra dicerminkan untuk membentuk area *neighborhood* tambahan sehingga pola intensitas di sekitar tepi citra tetap lebih terjaga dan artefak gelap yang dapat muncul pada *zero padding* dapat dihindari.

b. image_02.png

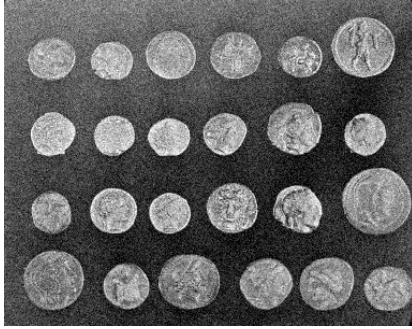


Citra *Astronaut* di atas memiliki *noise* berupa *salt-and-pepper noise*. *Noise* tersebut menyebabkan munculnya piksel dengan intensitas yang jauh berbeda dari piksel di sekitarnya. Oleh karena itu, digunakan metode *Median Filtering* untuk mengurangi *noise* tersebut. *Median filter* dipilih karena nilai median lebih tahan terhadap piksel dengan intensitas ekstrem (*outlier*). Dengan demikian, piksel *noise* dapat ditekan tanpa terlalu mengaburkan tepi dan detail objek pada citra.



Berdasarkan hasil di atas, dapat dilihat bahwa *Median Filtering* dengan kernel 3×3 berhasil mengurangi sebagian besar *noise* sehingga citra terlihat lebih bersih, sementara detail objek seperti wajah dan pakaian astronot masih dapat dipertahankan. Pada histogram, rentang penyebaran intensitas piksel secara keseluruhan relatif tetap, tetapi keseluruhan batang histogram menjadi lebih tinggi karena piksel *noise* digantikan oleh nilai median dari piksel di sekitarnya. Kernel 3×3 dipilih karena memberikan keseimbangan antara pengurangan *noise* dan pemeliharaan detail citra. Meskipun masih terdapat sebagian *noise*, peningkatan ukuran kernel menjadi 5×5 menyebabkan proses penyaringan menjadi terlalu kuat sehingga detail pada citra menjadi lebih buram dan beberapa bagian objek dapat kehilangan informasi. Oleh karena itu, kernel 3×3 digunakan untuk mempertahankan detail citra sekaligus mengurangi *noise* yang terdapat pada citra.

c. image_03.png



Citra kumpulan koin di atas memiliki *noise* berupa *salt-and-pepper noise*. Namun, *noise* tersebut tidak terlalu mengganggu keterlihatan detail pada citra. Permasalahan yang lebih signifikan terdapat pada pencahayaan, yaitu beberapa bagian koin terlalu terang sehingga detail pada permukaannya kurang terlihat. Oleh karena itu, digunakan teknik *Gamma Correction* dengan nilai $\gamma > 1$ untuk menurunkan intensitas pada area yang terlalu terang sehingga detail permukaan koin dapat terlihat lebih jelas.



Berdasarkan hasil di atas, dapat dilihat bahwa tekstur dan ukiran pada permukaan koin menjadi lebih jelas setelah intensitas pada area yang terlalu terang diturunkan. Penurunan intensitas tersebut meningkatkan perbedaan intensitas antara bagian permukaan koin, bayangan ukiran, dan tepi ukiran sehingga *boundary* pada detail permukaan menjadi lebih tegas dan lebih mudah dibedakan. Dengan demikian, struktur dan kedalaman ukiran pada koin terlihat lebih jelas secara visual. Pada histogram, distribusi intensitas piksel terlihat lebih dominan pada area intensitas rendah setelah proses *Gamma Correction*. Nilai $\gamma = 1.8$ dipilih karena mampu menurunkan intensitas pada area yang terlalu terang secara cukup signifikan, tetapi tetap mempertahankan perbedaan intensitas pada area gelap sehingga detail permukaan koin tidak ikut kehilangan informasi.

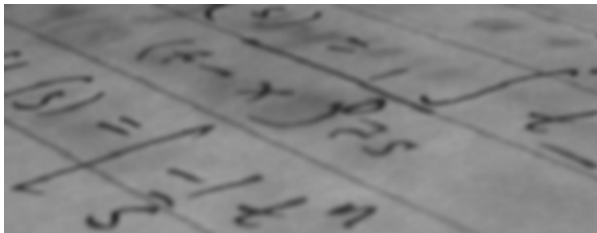
Adapun *noise* yang tersisa tidak perlu dihilangkan lebih lanjut karena sebagian besar berada pada area *background* yang kurang relevan terhadap objek utama. *Noise* pada

permukaan koin tidak terlalu mengganggu keterlihatan detail. Pemrosesan lebih lanjut dengan penerapan *Median Filtering* untuk menghilangkan *noise* tersebut justru dapat menyebabkan citra menjadi lebih buram dan mengurangi ketajaman detail pada permukaan koin, seperti yang ditunjukkan pada gambar di bawah ini.

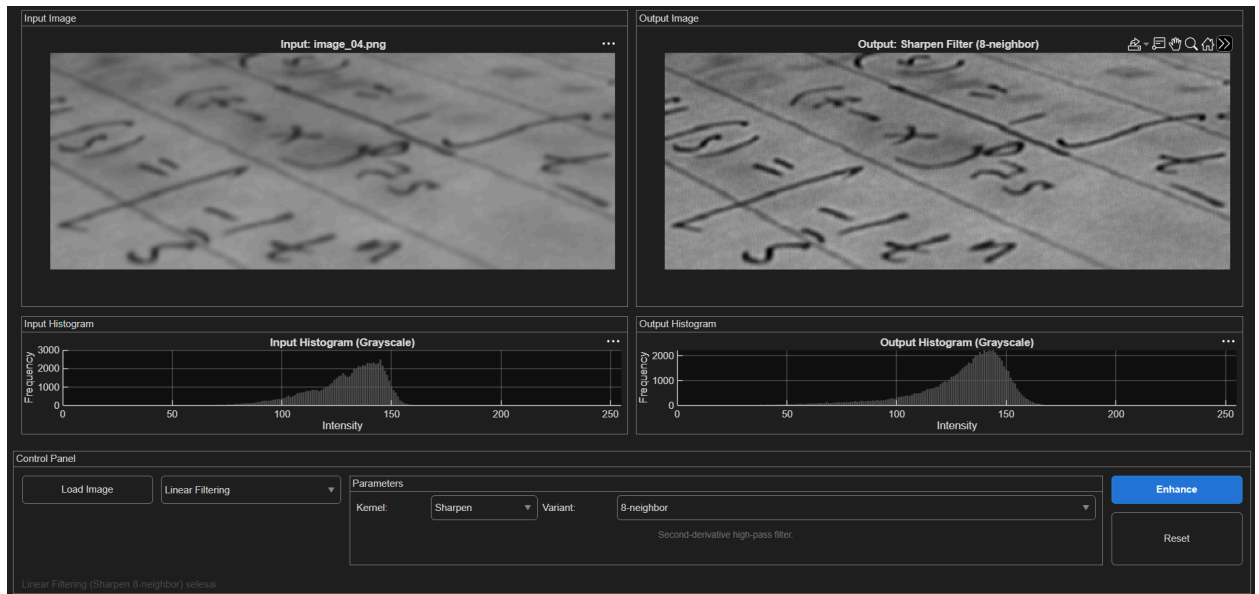


Oleh karena itu, *noise* yang tersisa lebih baik dipertahankan untuk menghindari penurunan kualitas detail pada objek utama.

d. image_04.png



Citra tulisan di atas tampak relatif buram pada keseluruhan area. Kondisi tersebut menyebabkan batas antara tulisan dan latar kertas kurang tegas sehingga keterbacaan tulisan menjadi berkurang. Perbedaan intensitas antarpiksel yang berdekatan pada batas tulisan dengan kertas (*boundary*) juga relatif kecil sehingga ketajaman visual citra menurun. Oleh karena itu, digunakan *Linear Filtering* dengan filter *sharpening* untuk meningkatkan perbedaan intensitas pada area tepi dan memperjelas detail tulisan.



Filter *sharpening* dipilih karena dapat memperkuat komponen frekuensi tinggi pada citra, terutama pada area yang memiliki perubahan intensitas secara tajam seperti batas antara tulisan dan kertas. Proses ini meningkatkan perbedaan intensitas pada *boundary* sehingga tulisan terlihat lebih tegas dan tajam dibandingkan citra asli. Adapun *8-neighbor* dipilih karena proses *sharpening* mempertimbangkan seluruh delapan piksel tetangga di sekitar piksel pusat sehingga perubahan intensitas pada tepi dari berbagai arah, termasuk diagonal, dapat diperkuat dengan lebih baik. Pada histogram, proses *sharpening* menyebabkan distribusi intensitas menjadi lebih tersebar mulus karena sebagian piksel di sekitar tepi mengalami peningkatan atau penurunan intensitas. Hal ini meningkatkan variasi intensitas lokal pada citra sehingga detail dan batas tulisan menjadi lebih mudah dibedakan secara visual.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil implementasi dan pengujian, telah berhasil dibuat aplikasi MATLAB berbasis GUI untuk melakukan analisis dan perbaikan citra. Aplikasi menampilkan citra input, histogram input, citra hasil *enhancement*, serta histogram output. Tersedia beberapa metode dan parameter yang dibutuhkan sesuai kebutuhan citra.

Metode *enhancement* yang digunakan adalah Intensity Transformation, Histogram Equalization, Histogram Matching, Linear Filtering, dan Median Filtering. Hasil pengujian menunjukkan bahwa pemilihan metode perlu disesuaikan dengan karakteristik dan permasalahan pada masing-masing citra. Citra dengan kontras rendah dapat diperbaiki menggunakan Contrast Stretching atau Histogram Equalization. Pasangan citra dengan area yang gelap atau terlalu terang dapat ditangani menggunakan Gamma Correction. Sedangkan untuk citra dengan noise, dapat diselesaikan dengan menggunakan Median Filtering tanpa terlalu menghilangkan detail.

Pengujian juga menunjukkan bahwa perubahan citra dapat diamati melalui perbandingan visual dan perubahan histogram sebelum dan sesudah *enhancement*. Dengan demikian, analisis histogram dapat digunakan sebagai dasar dalam menentukan metode dan parameter *enhancement* yang sesuai untuk suatu citra.

5.2 Saran

Pengembangan selanjutnya dapat menambahkan lebih banyak pilihan parameter dan metode *enhancement* pada GUI agar pengguna dapat melakukan penyesuaian citra dengan lebih fleksibel. Selain itu, pengujian dapat dilakukan dengan membandingkan lebih banyak metode untuk memperoleh hasil yang lebih sesuai dengan karakteristik masing-masing citra.

DAFTAR PUSTAKA

- [1] Munir, Rinaldi. 2025. "Histogram Citra".
<https://informatika.stei.itb.ac.id/~rinaldi.munir/Citra/2025-2026/06-Image-Histogram-2025.pdf>
- [2] Munir, Rinaldi. 2025. "Penapisan Citra dan Konvolusi".
<https://informatika.stei.itb.ac.id/~rinaldi.munir/Citra/2025-2026/07-Konvolusi-2025.pdf>
- [3] Munir, Rinaldi. 2025. "Image Enhancement (Bagian 1)".
<https://informatika.stei.itb.ac.id/~rinaldi.munir/Citra/2025-2026/08-Image-Enhancement-Bagian-1-2025.pdf>
- [4] Munir, Rinaldi. 2025. "Image Enhancement (Bagian 2)".
<https://informatika.stei.itb.ac.id/~rinaldi.munir/Citra/2025-2026/09-Image-Enhancement-Bagian-2-2025.pdf>
- [5] Munir, Rinaldi. 2025. "Image Enhancement (Bagian 3)".
<https://informatika.stei.itb.ac.id/~rinaldi.munir/Citra/2025-2026/10-Image-Enhancement-Bagian-3-2025.pdf>
- [6] MathWorks. 2026. "MATLAB GUI".
<https://www.mathworks.com/discovery/matlab-gui.html>
- [7] MathWorks. 2026. "Image Processing Toolbox Documentation".
<https://www.mathworks.com/help/images/>

LAMPIRAN

1. Repository GitHub Berikut adalah pranala ke repository GitHub yang berisi kode sumber dan dokumentasi proyek ini:

<https://github.com/Abrar-Abhirama/ImageEnhancement-GUI.git>

2. Pembagian tugas:

No.	NIM	Jobdesk
1.	13523081	Intensity Transformation
2.	13523081	Histogram Equalization
3.	13523038	Histogram Specification / Matching
4.	13523038	Image Filtering with Masking
5.	13523038	MATLAB GUI Integration
6.	13523038 & 13523081	Custom Histogram Function
7.	13523038 & 13523081	Dataset Analysis
8.	13523038 & 13523081	Technique Selection
9.	13523038 & 13523081	Before/After Comparison
10.	13523038 & 13523081	Laporan
12.	13523038 & 13523081	Final Testing & Debugging